主题汇总

RSI（自进化）方法汇总：七篇论文到底改了什么、怎么迭代

跨篇汇总 · 约 43 分钟

GitHub 源 ↗

关于这份文档

本仓库已读的七篇 RSI / 自进化方向论文，方法部分普遍公式与概念密集，读完后往往说不清**它到底改了什么、每个组件干什么、一轮迭代怎么转、轮与轮之间更新了什么**。这份文档把这七篇的方法按同一套问题重写一遍：详细、简洁、准确，概念落到具体载体（文件、代码、状态、门控），不罗列术语。

编号	论文	本文采用的版本	对应笔记
00220	Harness²: Recursive Agent Harnessing for an Open World	arXiv:2610.14224 **v1**（37 页，2026-10-09；唯一版本；README 该行指向 arXiv 官方页）	note.md
00218	HarnessPro: Recursive Harness Self-Improvement with Pareto Optimization for Performance-Cost Trade-offs	arXiv:2610.26770 **v2**（41 页，2026-10-20；v1 2026-10-19；README 该行指向 arXiv 官方页）	note.md
00214	Dream-RSI: Recursive Self-Improvement through Evolving Worlds	arXiv:2609.14858 **v2**（43 页，2026-10-06；README 该行指向 arXiv）	note.md
00212	RRSI: Regularized Recursive Self-Improvement of Agent Harnesses	arXiv:2609.24972 **v3**（27 页，2026-10-04）	note.md
00205	Prime Agent: A Self-Improving RLM Harness	arXiv:2608.23552 **v1**（16 页，2026-08-24；唯一版本）	note.md
00199	HarnessX: A Composable, Adaptive, and Evolvable Agent Harness Foundry	arXiv:2606.14249 **v3**（43 页，2026-07-23；最新版；README 该行指向 arXiv）	note.md
00189	HarnessDev: Can LLMs Create and Evolve Their Own Agent Harness?	arXiv:2609.01437 **v1**（41 页，2026-09-01；唯一版本；README 编号单元格指向上游归档 PDF，与该 v1 字节一致）	note.md

核对方式（生成日期 2026-10-15；2026-10-17 增补 HarnessX；2026-10-26 增补 HarnessPro；2026-10-29 增补 Harness²）：本文所有机制描述与数字均按上表版本**回原文核对（核心正文/表格与所引附录关键段逐项；附录 D 完整 listing 未逐行）**；逐篇〈关键数字〉里的每个数字都带出处（页/表/图/节），速览表与坐标系表只给结论，详细出处见对应小节。原文内部自相矛盾之处按「表格/附录值优先于摘要值」处理，并在对应小节写明。个别无法核实的点就地标注。七篇原文的核对手段与限制见文末〈版本与核对说明〉。

约定：

- 正文段落、列表、表格中**不写行内公式**（GitHub 渲染不可靠）；全篇只有 3 处独立公式块（用 `math` 代码围栏——GitHub 上比列表内的 `$$` 稳定；都在方法的核心处），都紧跟一句白话读法。
- 伪代码与协议用围栏代码块；迭代闭环用 Mermaid 图（GitHub 原生支持）。
- 同一个词在不同论文里含义不同的情况，统一放到〈概念对照〉一节消歧；先读那一节也能帮助理解各家方法。

一句话预告：这七篇的「自我改进」其实是七种不同的东西——改 **harness 源码**（RRSI）、改 **harness 的持久状态**（Prime Agent）、改 **探索策略代码**（Dream-RSI）、把 **harness 配置做成可组合/可演化的一等对象并接上模型共演化**（HarnessX）、**测量"模型能不能自己造并改 harness"这件事本身**（HarnessDev）、**把执行成本接进 harness 演化的选择器**（HarnessPro）、以及 **在没有标准答案的任务流里同时改执行 harness 和"改进 harness"的经验库**（Harness²）。看清这一点，七篇的方法就不再互相混淆。

一页速览

编号 / 论文	被进化的东西	一次迭代是什么	谁动手	产物与载体	反馈信号	接受规则 / 何时停
00220 Harness²	执行 harness（8 个文件级组件）+ 改进 harness 里的经验库	k 次参考执行 → INV/DOM 候选各 k 个 → 每个候选探测一次 → k 个合成器各合出一份后继并执行 → 匿名终选（k>1 才有）	executor 冻结；editor 与 selector 明示为 Claude Opus 4.8（合成/反思同款，原文未单独点名）	harness 文件树；库条目 = harness 树 + `EXPERIENCE.md` + 索引	参考执行 vs 候选执行的行为对比（改进期不接 verifier、不读分数）	无分数门槛：合成器按证据合并/取舍/保留原状；任务内预算 k 预先固定；任务间不设停止条件

编号 / 论文	被进化的东西	一次迭代是什么	谁动手	产物与载体	反馈信号	接受规则 / 何时停
00218 HarnessPro	完整 harness（instructions / tools / skills / context/memory / hooks / verification policies）	G=3 rollouts → analyzer 诊断 → planner 三份计划（I1/I2/I3）→ editor 每个父候选 3 个（最多 9 个）→ D_val 固定生成测试评估 → Pareto 过滤 + 归一化加权距离选三档	analyzer / planner / editor（LLM agents）；test synthesizer 只提供评估器；选择是算法	harness 源码 + Git commit；固定测试 S(x)；lite/medium/heavy 三档	固定 LLM 生成测试的 pass rate + token 成本（label-free）	PR_val ⩾ PR_val(h0)、Pareto 非支配、每档 J_m 最小；每轮最多 3 个；T=3/4 epochs；最终 held-out test
00214 Dream-RSI	探索策略（一份可执行的 Python 代码）	在线跑出一棵新发现树 → 离线在历史树上回放多个候选策略版本	policy-development agent（LLM 改写策略代码）	新的策略代码；记录树**只增不改**	回放分（基于树里已记录的真实评测分）	每轮取回放分最高者；外层轮数：Lasso 5、kernel 与 harness 9、数学 10
00212 RRSI	harness 源码（9 类组件都可增删改）	提议 2 个候选 → 泄漏筛查 → 全量 evolve set 评测 → 过三道非补偿准入条件 → 择优替换	proposer LLM 起草候选，算法选择（policy 全程冻结）	harness 源码 diff；编辑台账	有限 evolve set 上的分数与 token 成本	泄漏筛查 + 三道准入条件全过才可替换；无合格者原地不动；20 / 20 / 40 轮
00205 Prime Agent	harness 的持久状态（L3 四类条目）	轨迹证据 → refinement → 轮次边界应用、版本化 → 组装进下次调用的补充 prompt	agent 自己（直接请求，或运行 `/refine` 开后台模型调用）	版本化条目：prompt notes / memories / skills / subagent specs；**模型权重不动**	执行轨迹证据本身（没有 verifier 模型、也没有 reward）	无量化门槛；条目可增删改、可回滚；不改写不可变的基础 prompt
00199 HarnessX	harness 配置（模型角色 + processor / prompt / 工具 / 控制流；共演化时还包括模型权重）	每轮在适应批次上 rollout → Digester / Planner / Evolver 提议 → Critic + 确定性门 ship / reject	AEGIS 的 meta-agent（Opus 4.6）；**确定性门拥有最终否决权**	类型化 harness 配置 + change manifest；共演化时共享 replay buffer	完整 trace + 固定 verifier 打分（按 pass@2 聚合）	每轮最多 4 个候选（实验设置）、最多 15 轮；连续 3 轮无 ship 即停
00189 HarnessDev	模型自建的 harness 代码（被测量对象）	创建者在 git 工作区改代码 → 成对提交 → 双基准全量评测 → 读逐任务反馈	创建者模型，在现成开发环境（Claude Code / Codex）里做普通 coding 工作	harness 源码 + 控制器账本（冻结快照、事件流、预算计数）	100 个 SWE-Pro + 89 个 Terminal-Bench 任务的全量分数	预算 10 对完整评测；探针每轮≤2 次；创建者 `declare-final` 申报；事后 630 留出实例审计

00220 Harness²：把「改进 harness」本身也做成可递归的对象

> 版本：arXiv **v1**（37 页，2026-10-09；截至 2026-10-30 只有 v1）。页底注明第一作者 Ruiyao Xu 是在 Google Cloud AI Research 实习期间完成本工作；9 位作者中 8 位属 Google Cloud AI Research，清单按团队归属列入 Google 组。

一句话

任务内改执行 harness、任务后改「改进 harness」：前者用「提议（Propose）–探测（Probe）–合成（Compose）」在没有标准答案的条件下靠**行为对比**改文件；后者把被采纳的 harness 与改进过程中的经验写进一个**库**，让下一次改进有东西可参考。模型权重、改进用的阶段提示词与决策规则全程冻结，变的只有库。

它要解决什么

开放世界里的 harness RSI 有两个具体障碍（§1，p2）：

1. **没有可靠 verifier**：任务在 harness 设计阶段没见过、测试时是全新的，没有 ground truth 或验证反馈；已有方法靠代理信号、假定答案或两两输出比较来指导改动。
2. **改进过程本身不积累**：为一个任务改好的 harness 高度专用，换需求就废；直接把它继承到下一个任务又会引入噪声。作者的说法是——改动的**过程**（怎么诊断、试了什么、执行揭示了什么）其实高度可复用，但没人把它当成学习对象。

论文给的负面证据（Table 1/2、Figure 7 左）：ReasoningBank 在编码任务上最低比 base 低 6.6 分；Harness Scaling 在 12 个 benchmark–配置组合里有 7 个低于 base；而"直接改 harness、既不跑参考执行/探测、也不读库"（Figure 7 左的 "Direct Editing w/o Execution"）会让 WB-code 从 73.4 掉到 63.2。

系统由哪些部件组成

部件	具体是什么	作用	输入 → 输出
执行 harness H	8 个文件级组件：C1 `systemprompt.md`、C2 `guardrails.md`、C3 `LongTermMEMORY.md`、C4 `tool_guidance.md`、C5 `sub_agents/<n>/AGENT.md`、C6 `skills/<n>/SKILL.md`（可带 `scripts/`）、C7 `scripts/<name>.py`、C8 `plugin/<name>.ts`（Table 5，p22）。沿用 AHE 的分解并**多加插件这一层**	被改进的对象；每个组件是一个文件，所以一次编辑就是一次 diff	任务 → 轨迹与交付物
改进 harness R	固定的阶段提示词（编辑者、合成器、终选器、反思器）+ **可变的库**	指导"怎么改"；其中只有库在任务之间变化	当前任务证据 + 库 → 提议/合成/选择
库（每个 benchmark 域一个）	条目 = 一份完整被采纳的 harness 树 + 一份 `EXPERIENCE.md`（适用情境、经验、支撑证据）+ 索引	跨任务复用的载体：组件可被复制/改写，经验指导怎么用	采纳的 harness + 改进记录 → 条目
Harness Editor（两个 scope）	INV：怎么干活（流程、工具、上下文、委派）；DOM：这一行的从业者知识（交付物、口径、检查项）	各提 k 个候选；两个方向**分开提**（Prompt 1/2 要求各自只改自己那一类、不要越界）	任务、当前 harness、参考执行、历史、库 → 候选 harness
Harness Composer	并行：k 个独立会话拿同一个候选池各自合成；顺序：链式 checkpoint	选择性合并/修订/采纳/保留；每条决策要绑到轨迹里的具体片段	参考执行 + 候选及其 diff → 后继 harness
Selector	对**匿名**交付物做终选（看不到输出来自哪个 harness，也不能改写）	选出提交哪个输出；被采纳的记作 H*（"被采纳"，不是"最优"）	n 份交付物 → 一份
Experience Reflector	逐组件写 edits 表（helped / hurt / no observed effect / confounded），最多写一条经验	把"改动 → 行为差异"落成可审计的记录	完整改进记录 → 库更新
一道 guard	拒绝把含 grader verdict、rubric 或 score 的工作目录交给任何会话	保证"无标签"不是口号	工作目录 → 放行/拒绝

一次迭代怎么走



1. **参考执行**：当前任务先用 H 跑 k 次（主表 k=1 时是 1 次），得到"这套 harness 现在怎么行为"的基线（§2.2.1，p4）。
2. **提议**：编辑者按 INV / DOM 两个 scope 各提 k 个候选（k=1 时各 1 个），每个候选都是**直接改好的可执行文件树**，不是文字建议。提议会参考库里的组件，但工作区事实必须回到当前任务证据去核实（p21、Prompt 1）。
3. **探测**：每个候选在同一任务、同一模型与环境配置下各跑一次，与参考执行做**行为对比**——是缓解了目标缺陷、没变化，还是引入回退。作者强调这不是在验证正确性。
4. **合成**：k 个合成会话各自读全池（参考执行、2k 个候选及其 diff），独立合成一份后继 harness 并执行；也可以 KEEP_BASE。**并行递归下 k 控制宽度**（只做一次种群更新），而顺序递归里 k 就是深度：$H^{(0)}=H_0$，每次把一个对比步的产物当作下一步的输入，链式做 k 步（§2.2.2，p6）。
5. **终选**：k>1 时由选择器在匿名交付物里挑一份，不改写。
6. **反思与更新**：任务结束后回看完整记录 D_t，把采纳的 harness 与经验写进库；下一个任务**仍从 H_0 出发**，只是提议阶段能读库。

轮与轮之间更新了什么

更新什么	载体	谁写	何时写	可否回滚
被采纳的 harness + 经验	库条目（harness 树 + `EXPERIENCE.md` + 索引），按 benchmark 域分开	Experience Reflector	每个任务结束后	**没有删除/修订机制**：证据不足时只是不新增条目（"NO_ENTRY"），§G 把合并/修订列为未来工作
执行 harness	每任务内部：H_0 → 候选 → 后继	Composer	任务内	任务结束即弃用；不跨任务继承
不更新：模型权重、阶段提示词、决策规则	—	—	—	p7 原文："Model weights, stage instructions, and the decision rule remain fixed, while the library evolves."

这也是它和 RRSI / HarnessPro 最容易被混淆的地方：**第二级"学习"不是学会更好地改，而是记住"哪些改法和经验可复用"**。

什么时候停、什么算「更好」

- **任务内**：预算 k 在实验开始前固定（主表 k=1，`+ Recursion` 用并行 k=5）；**没有分数阈值，也没有非补偿的准入条件**——合成器按证据支持多少改多少，证据不足就保留原 harness。唯一的硬门是 harness 的合法性校验：skill 必须有触发句、always-on 文本不能有 Jinja 花括号、被引用的脚本

必须随附（p21 的 validator 重试机制、Prompt 1，pp.32–33；三条规则原文用 "THREE RULES THAT DISCARD THE WHOLE PROPOSAL" 标出）。

- **终选**：只挑不改，H* 表示"被采纳"而非"被验证最优"（§2.2.2，p6）。
- **任务间**：没有停止条件，任务流结束即止；也没有"回放已完成任务"（§5、Appendix A.1，p18）。
- **算不算更好**：靠 benchmark 自己的评分器事后判定；运行期没有任何"分数上升"的信号可用。

监督信号从哪来

- **行为对比**（contrast）：同一任务上，参考执行与候选执行的轨迹/交付物差异。它不是正确性信号，只是一种"改动有没有改变行为、改变了什么"的证据（§2.2.1，p5）。
- **模型判断**：editor 与 selector 明示为 Claude Opus 4.8（p20–21）；Appendix C 说 "Revision and reflection use Claude Opus 4.8, matching our editor"，即合成/反思也用同款（**合成阶段是否点名 Opus 4.8，原文没有单独写**）。反思器把每条改动标成 helped / hurt / no observed effect / **confounded**：只有 confounded 支撑的经验要么收窄、要么不写，整份记录没有具体可复用的执行对比时才写 NO_ENTRY（Prompt 6, pp.36–37）。
- **防泄漏**：编辑者/合成器/选择器的上下文里没有 rubric、答案或分数；工作目录由 guard 过滤；benchmark 分只用于评测（Appendix A.1、D.2）。
- 作者明确承认：单条轨迹（Prompt 3 明说 "Each candidate was rolled out ONCE"）、执行随机性与模型共同偏差都会削弱证据强度（§2.2.1 p5、Prompt 3 p33、§F p29）。

关键数字

全部为 **v1**（Table 1 p8、Table 2 p9；三次乱序任务流的均值，括号内 SD）：

配置	WorkBuddy Avg.	JobBench	LAB Partial
Gemini 3.5 Flash / OpenCode	72.4 → 77.5（+5.1）	31.5 → 36.0（+4.5）	79.9 → 81.9（+2.0）
Gemini 3.5 Flash / Codex	74.5 → 77.5（+3.0）	33.9 → 35.3（+1.4）	71.6 → 75.6（+4.0）
GLM-5 / OpenCode	64.2 → 75.7（+11.5）	20.6 → 24.2（+3.6）	59.6 → 71.4（+11.8）
GLM-5 / Codex	64.4 → 75.3（+10.9）	23.6 → 24.8（+1.2）	58.1 → 65.9（+7.8）

（差值为本文复算，与摘要的 "up to 11.5 / 4.5 / 11.8" 一致。）

- **任务级递归再加**：并行 k=5 相对 k=1 再涨 2.9–3.9（WorkBuddy 的 Avg 口径；分域最大 +5.3）、3.0–4.2（JobBench）、2.6–6.7（LAB Partial）（Table 1/2 的 `+ Recursion` 行）。
- **消融**（Table 3，p10，Gemini 3.5 Flash/OpenCode，k=1）：去掉库 −3.1（WorkBuddy）/−1.8（JobBench），去掉 INV/DOM 分工 −2.9/−3.7，去掉合成 −1.6/−1.9。
- **组件消融**（Table 6，p23，从已改好的 k=5 harness 里拆、不重跑改进）：Avg 上 prompt 组 −1.5、skills −5.3、scripts −3.7；Office 对 scripts 最敏感（−7.4）。
- **库的作用**（Figure 7 左，p11）：这一栏是 "Direct Editing w/o Execution"——直接改 harness，**既不做参考执行/探测，也不读库**（与 Table 3 的 "w/o library" 不是同一实验，后者只去掉库、仍做对比式改进）：Code 73.4→63.2、Web 69.9→65.0（Office 反而 74.0→77.1）；k=1 库回到 73.8/72.6/77.5，k=5 库带来 +0.9/+4.4/+5.1。
- **库的迁移**（Figure 7 右、Figure 11，p11/p24）：JobBench 库迁到 WorkBuddy 再 +0.6/+0.5/+2.9；留一法重跑 77.5→79.4、36.0→37.7（作者注明是离线复用分析）。
- **同模型自改**（Figure 8，p11，k=2）：5 个模型在 3 个 benchmark 上全部上涨；Opus 5 +3.3/+5.5/+3.0，GLM-5 在 LAB 上 +13.7。
- **与生产 harness 比**（Figure 13，p25）：从 OpenCode 出发的 k=5 比每域最强的固定 harness 再高 4.6/5.8/4.9，比自己出发时高 7.4/8.4/9.4。
- **工具可靠性**（Table 7，p25，k=1）：错误率 0.8%→0.5%、14.4%→9.1%、2.0%→1.6%；调用次数在 WB 降（20.0→17.8），在 JobBench/LAB 升（27.4→32.8、20.8→25.2）。
- **成本**：k=1 是 1 次参考 + 2 次探测 + 1 次合成后执行；k=5 是 5 + 10 + 5（**按 §2.2.2 的机制推算，原文没有给这条成本公式**）；Figure 6（p10）画的是每任务总 token（含改进）与增益，对数横轴，两种 k 都在经验 Pareto 前沿上；**没有墙钟数据**。

陷阱与边界

- **没有任何自动准入/回滚机制**：与 RRSI 的三道非补偿条件、HarnessX 的确定性门不同，这里**采纳什么 harness 由 composer 的合成决策决定，selector 只决定哪份交付物对应 H***；失败案例（Figure 14，p26）的 9 个回退里有 6 个属于"选错候选"或"改动写错了前提"两栏（例如 96→55、100→70、54→26）。

- **第二级递归的"学习"很窄**：更新对象只是库，提示词与决策规则冻结（p7）。
- **统计强度弱**：只报均值与 SD（三次运行），没有置信区间或显著性检验；JobBench 上 36.0 对 35.8（SD 2.3 vs 1.0）只是均值意义上的领先。
- **judge 与 editor 同源**：JobBench 的评分模型就是默认编辑器 Claude Opus 4.8（Appendix B，p19）；LAB 用 Claude Sonnet 4.6，只取 1,251 题里的 100 题。
- **顺序依赖**：任务在域内顺序执行、库随流增长，作者用三次乱序流取均值缓解，但没有报告顺序方差分解。
- **单条轨迹当证据**：每个候选只跑一次；提示词里专门写了"一次好或坏都是弱证据"。
- **库只增不减**：长流下会膨胀（§G 把管理列为未来工作）。

00218 HarnessPro：把「执行成本」接进 harness 演化的选择器

> 版本：arXiv v2（41 页，2026-10-20；v1 2026-10-19）。v1→v2 的差异是第 2 单位 `Z.AI Co., Ltd.` → `Z.AI`、第 5 单位 `Kuaishou-Kuaishou Technology` → `Kling AI`、补共同一作/通讯作者标记；正文/Table 2 数字不变。v2 PDF 在 2026-10-25 与 2026-10-26 两次抓取得到不同副本：旧副本（sha `ce88…`）p4 Figure 1 不完整，当前副本（sha `c633…`）完整；两版除 p4 外逐页文本一致（见〈版本关系〉）。

一句话

把 harness RSI 从「只追 pass rate」改成「pass rate + 每任务执行成本」的双目标搜索：用固定的 LLM 生成测试替代 LLM 直接打分（label-free），让同一 planner 按省成本 / 平衡 / 保性能三种偏好各出一份计划、editor 实现成候选，再用 Pareto 过滤 + 归一化加权平方距离选出 lite / medium / heavy 三个部署档位，递归多轮；最终只在 held-out test 上评测。

它要解决什么

- 已有 harness RSI（RHO、AHE 等）主要优化 pass rate；更多探索/验证提升性能，但推高 token 成本与延迟（§1，p.1）。
- RHO 的 label-free 选择靠 LLM judge 自偏好；作者发现 LLM judge 对执行轨迹**过于乐观**，常把有缺陷的实现判成成功（§4.3，p.6）。
- 两个挑战（§3 末，p.3）：harness 组件（instructions、tools、skills、hooks、rules）组合空间巨大且离散；性能与成本必须同时权衡。
- 本文的取舍：演化期不用 ground truth、官方测试或 grader；用固定生成测试评估候选；用 Pareto 选三档而不是单一 harness；Table 1（p.3）把定位压成「Pass + Cost / Harness / Label-free / Recursive / Pareto set」。

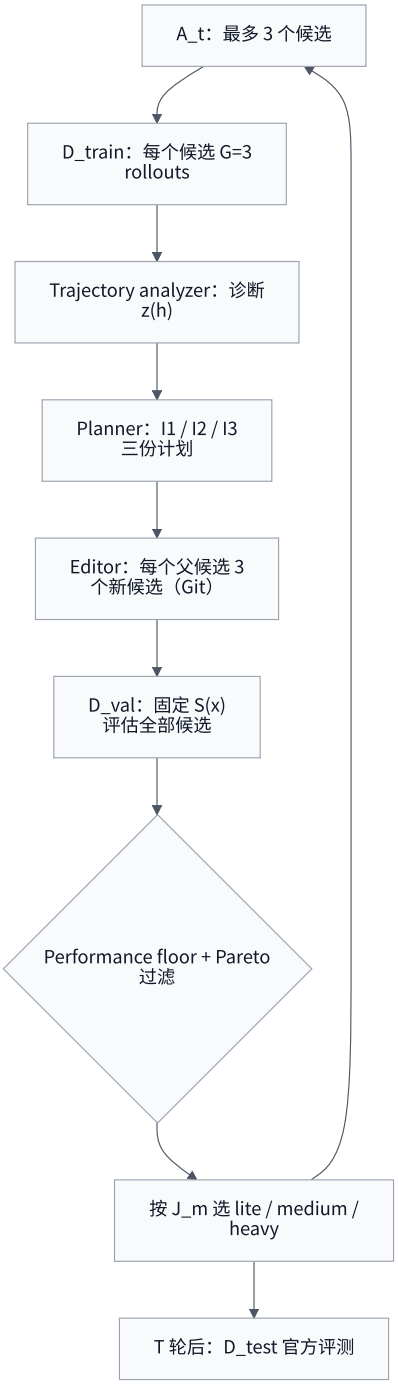
系统由哪些部件组成

部件	具体是什么	作用	输入 → 输出
DPP coreset	用 DPP 核从训练任务里选 10 个任务；难度分数 + 失败模式指纹，ε=0.1、θ=0.7；演化前选一次、全程固定（§4.1、§A.1，p.4/p.13）	控制每轮 rollout/诊断的规模，兼顾难度与多样性	训练任务 → 10-task D_train
候选集 A_t	第 t 轮的 harness 集合；A_0={h_0}；每轮最多保留 3 个（§4.1、§4.3，p.4/p.6）	演化搜索的种群；至多三个代表进入下一轮	上一轮选择 → 当前候选
trajectory analyzer	LLM agent；联合看每个候选在每个训练任务上的 G=3 条 rollout，输出 problem / impact / source / evidence（轨迹文件路径 + 事件区间）；跨任务聚合 z(h)；超过 64K token 时摘要（§4.2，p.4–5）	把长轨迹压成可执行的诊断	轨迹 + 指标 → 诊断 z(h)
evolution planner	LLM agent；按 I1（省成本）/ I2（平衡）/ I3（保性能）三种 instruction 各出一份计划；看诊断、指标、候选间 diff；默认不看完整轨迹；D_val/D_test 对它隐藏（§4.2、§B，p.5/p.15–16）	把诊断变成三份可执行的修改计划	诊断 + 指标 + diff → 计划 p_j(h)
harness editor	LLM agent；在独立 workspace 实现计划，commit 到 Git；每个父候选产生 3 个新候选（§4.2，p.5）	把计划落成可执行 harness	计划 + 诊断 → 候选 harness
test synthesizer	LLM agent；对每个 D_train ∪ D_val 任务生成 10 个可执行测试 S(x)；用冻结的 Empty 产物做执行诊断；MCP 工具在 fresh container 里运行；无官方测试/grader（§4.3、§C.3，p.6/p.22–23）	提供 label-free 的固定评估器	任务描述 + 初始轨迹 → 固定测试 S(x)
评估/选择	在 D_val 上每个候选每任务评估一次；先按 PR_val(h0) 设 floor、再做 Pareto 过滤；每档用 J_m 选代表（§4.3，p.6）	把候选压成 lite/medium/heavy 三个	候选 + S(x) → 三档代表
运行时产物	三种 benchmark 的 runtime 产物合计含 model instructions / policy.json / hook.py；TB2/SWE 另有 profile rubric，SWE 另含 repair-verify / checklists；ProgramBench medium 用固定 high-effort profile、无 routing rubric（§D，p.24–41）	部署时的路由、预算、压缩、验证	任务描述 → 执行配置
冻结的 backbone	GPT-5.5 × Codex 0.147.0；GLM-5.2 × Claude Code 2.1.231；模型权重冻结（Figure 1，p.4；版本见 §5.1，p.7）	执行任务的模型；演化只改 harness	任务 + harness → 轨迹
成本模型	按 uncached input / cached input / output token 的公开价格计（Table 4，p.14）	把 token 用量换成每任务美元成本	token 计数 → 成本 C(h)

一次迭代怎么走

外层第 t 轮（§4、Algorithm 1，p.4–6）：

1. 在 D_train 的每个任务上，对 A_t 的每个候选跑 G=3 条 rollout，记录 pass / cost / latency 与轨迹。
2. analyzer 对每个候选聚合 3 条轨迹，输出诊断 z(h)。
3. planner 对每个候选按 I1/I2/I3 各出一份计划；editor 在独立 workspace 实现，Git commit；C_t = 每个父候选 3 个新候选，$\tilde{C}_t = A_t \cup C_t$。
4. 在 D_val 上用固定的 S(x) 评估 $\tilde{C}_t$ 的每个候选（每任务一次），得到 PR_val / C_val / L_val。
5. 过滤：低于 PR_val(h0) 的、被 Pareto 支配的候选丢掉；对剩余候选按 Eq 8 归一化、按 Eq 9 为三档各选一个代表（可能重合，实际至多三个不同 harness）。
6. A_{t+1} = 最多三个代表（过滤后可能少于 3 个不同 harness）；t←t+1；回到 1。
7. T 轮后冻结，只在 D_test 上跑官方评测。



轮与轮之间更新了什么

更新什么	载体	谁写	何时写	可否回滚
候选集 A_t	第 t 轮保留的至多三个 harness	选择算法（Eq 9）	每轮 D_val 评估后	论文没有把它写成正式的回滚机制；父候选仍在 C̃_t 里参与比较
harness 源码 / 配置	Git 工作区 + commit	editor	每轮计划后	Git 可追踪/回退（§4.2）；论文未把它写成正式的回滚机制
诊断与指标	analyzer 输出、每轮 metrics	analyzer / 评估	每轮	只作为下一轮输入，不是持久状态
固定测试 S(x)	每任务的 10 个测试	test synthesizer	演化前一次生成，之后固定	不更新（保证候选可比）
模型权重 / 采样	GPT-5.5 / GLM-5.2	—	全程不动	—
D_val / D_test	任务划分	—	全程不动；planner 不可见	—

什么时候停、什么算「更好」

- **停止**：epoch 上限 T=3（Terminal-Bench 2 / SWE-Bench Pro）或 4（ProgramBench）；论文没有早停规则；每轮最多保留 3 个，过滤后不足 3 个就少于 3 个（§5.1、Figure 3 脚注 2，p.7/p.6）。
- **准入**：PR_val(h) ⩾ PR_val(h0)（性能地板）；且不被 C̃_t 里其他候选 Pareto 支配（Eq 7，p.6）。
- **三档选择**：对剩余候选算 d_perf / d_cost（Eq 8），每档用权重 (α_m, β_m) 算 J_m（Eq 9），选最小；lite=(1,2)、medium=(1,1)、heavy=(2,1)（§A.2，p.13）。

$$d_{\text{perf}}(h) = \frac{P_{\max} - \text{PR}_{\text{val}}(h)}{P_{\max} - P_{\min}}, \qquad d_{\text{cost}}(h) = \frac{C_{\text{val}}(h) - C_{\min}}{C_{\max} - C_{\min}}, \qquad J_m(h) = \alpha_m d_{\text{perf}}(h)^2 + \beta_m d_{\text{cost}}(h)^2$$

  读法：先把 pass rate 与成本各自按当前候选池的最大/最小值归一化到 0–1，再按模式权重算加权平方距离；lite 更重成本，heavy 更重性能。
- **最终评测**：T 轮后冻结，只在 D_test 上跑官方测试；演化期的选择只用 D_val，不看 D_test（§4.3，p.6）。

监督信号从哪来

- 固定 LLM 生成测试 S(x)：每个 D_train ∪ D_val 任务 10 个可执行 case；从任务描述与初始 harness 的轨迹生成；之后全程固定；无 ground truth、官方测试或 grader（§4.3、§C.3，p.6/p.22–23）。
- 测试质量：Figure 2（p.6）在 TB2+SWE 训练任务上汇总，官方 Fail 12 个中，LLM judge 判 Fail 5 个（判 Pass 7 个；failure recall 41.7%），生成测试判 Fail 10 个（判 Pass 2 个；83.3%）；官方 Pass 8 个中生成测试误判 1 个（12.5% 假阳性）。正文只强调 recall，未给 precision；若以 Fail 为正类，由混淆矩阵复算 failure precision=10/(10+1)≈90.9%。
- 成本：按 token 价格（Table 4，p.14）计；延迟是辅助指标，不计 grading 时间（§3、§5.1）。
- 划分：D_train=DPP 10 任务，D_val=其余训练任务，D_test=held-out（59/100/100）；planner 对 D_val/D_test 不可见（§4.2、§5.1）。

关键数字

事实	数字	出处
实验设置	3 benchmark：Terminal-Bench 2（留出 59）、SWE-Bench Pro（100）、ProgramBench（100）；D_train=10-task DPP coreset；每个候选每训练任务 G=3 次 rollout；epoch 3 / 3 / 4；每轮最多保留 3 个；GPT-5.5 × Codex 0.147.0、GLM-5.2 × Claude Code 2.1.231；TB2/SWE 报 3 次运行、PB 报 2 次；ProgramBench 的 P 是每任务测试通过比例的宏平均（Table 2 题注，p.8）	§5.1，p.7；§A.1，p.13
Heavy vs RHO（GPT-5.5 × Codex，P / C / T）	TB2 80.8±1.0 / 0.72±0.05 / 310±31 vs 76.3 / 1.09 / 354（+4.5 pp，−33.9% 成本）；SWE 80.7±0.6 / 1.97±0.02 / 333±10 vs 80.0 / 2.68 / 437（+0.7 pp，−26.5%）；PB 69.6±1.03 / 2.95±0.13 / 888±32 vs 68.8 / 3.59 / 928（+0.8 pp，−17.8%）	Table 2，p.8；Δ 为复算
Heavy vs RHO（GLM-5.2 × Claude Code）	TB2 81.4±1.7 / 0.62±0.02 / 1080±15 vs 78.0 / 1.00 / 1318（+3.4 pp，−38.0%）；SWE 77.3±0.6 / 1.01±0.03 / 412±11 vs 75.0 / 2.17 / 520（+2.3 pp，−53.5%）；PB 64.4±0.8 / 14.95±0.08 / 5925±110 vs 63.3 / 17.00 / 6434（+1.1 pp，−12.1%）	Table 2，p.8；Δ 为复算
摘要的平均数	GPT：+2.0 pp / −26.1%（三个 benchmark 逐项变化的算术平均：4.5/0.7/0.8 与 33.9/26.5/17.8）；GLM：+2.3 pp / −34.5%（复算：3.4/2.3/1.1 与 38.0/53.5/12.1）；按等权总成本复算 23.4%、按任务数 59/100/100 加权 22.7%	摘要 p.1；Table 2 p.8；复算
medium / lite	GPT medium：TB2 76.3 / 0.56、SWE 79.0 / 1.26、PB 67.9 / 2.90；GPT lite：72.9 / 0.43、72.3 / 0.80、64.3 / 2.18；GLM medium：76.3 / 0.50、75.0 / 0.87、63.0 / 13.74；GLM lite：75.1 / 0.43、70.7 / 0.70、60.1 / 11.10	Table 2，p.8
与 RHO+Iteration 比	GPT TB2：medium 76.3 / 0.56 与 heavy 80.8 / 0.72 / 310 对 RHO+Iteration 76.3 / 0.82 / 328（medium −31.7% 成本；heavy +4.5 pp、−12.2%）；SWE heavy 80.7 / 1.97 / 333 对 80.0 / 1.90 / 375（+0.7 pp，成本 +3.7%）；PB heavy 69.6 / 2.95 / 888 对 68.3 / 3.13 / 789（+1.3 pp，−5.8%）	Table 2，p.8；§5.2，p.7–8；Δ 为复算

事实	数字	出处
延迟	heavy 相对 RHO 六组均更低；GPT lite 在 PB 985 vs 928（更慢），GLM lite 3907 vs 6434（更快）	Table 2，p.8；§5.2，p.8
消融	去 planner：多数实验 pass 降/成本升（SWE medium 79.0 / 1.26→73.0 / 1.42；例外 TB2 lite 72.9 持平、0.43→0.41）；去 test synthesizer：TB2/SWE 三档 pass 降 3.4–10.0 pp（SWE medium 79.0→69.0，heavy 80.7→72.0）；CoP heavy 0.89 / 2.44 vs RHO 1.43 / 3.35	Table 3，p.9；§5.5，p.8–9
演化动态	TB2 heavy 74.6→80.8，成本 0.91→0.72；lite/medium 从 epoch 2→3 保持 pass、成本 0.61→0.43 / 0.68→0.56；SWE medium 70.0 / 1.57→77.0 / 1.66→79.0 / 1.26；PB 第 4 epoch 三档降成本、pass 大致持平	§5.4，p.8；Figure 4，p.9
测试合成器	官方 Fail 12 个：LLM judge 判 Fail 5/12（41.7%；判 Pass 7/12），生成测试判 Fail 10/12（83.3%；判 Pass 2/12）；官方 Pass 8 个：生成测试误判 1 个（1/8=12.5%）；正文未给 precision；若以 Fail 为正类，复算 failure precision=10/11≈90.9%	Figure 2，p.6；§4.3，p.6；复算
价格	GPT-5.5（<272K）uncached 5.00 / cached 0.50 / output 30.00 美元/M；GLM-5.2 1.40 / 0.26 / 4.40 美元/M	Table 4，p.14

陷阱与边界

1. **演化成本没有计入每任务成本**：Table 2 的成本是任务执行成本；T 轮 rollout、analyzer/planner/editor 的 LLM 调用、每任务 10 个测试的合成、每轮全 D_val 评估都没有报告。部署 TCO 可能明显更高（§3、§5.1、Algorithm 1 第 17 行）。
2. **摘要的 +2.0 pp / −26.1% 是逐项均值**：三个 benchmark 的 pass 差 4.5/0.7/0.8、相对成本降幅 33.9/26.5/17.8；按等权总成本复算 23.4%、按任务数加权 22.7%，不是池化值（Table 2，p.8）。
3. **heavy 的「低成本」是相对 RHO，不是相对 Vanilla 的每一项**：GPT 下 SWE 1.97 vs 1.88（更贵）、PB 2.95 vs 2.65（更贵）、TB2 0.72 vs 0.89（更便宜）；GLM 下 TB2 0.62 vs 0.92、SWE 1.01 vs 1.17（更便宜）、PB 14.95 vs 11.34（更贵）。§5.2 的 "better task performance need not come at a higher execution cost" 是 RHO 对比语境。
4. **Pareto 支配的读法**：§5.5 的「三个 RHO 变体都被 HarnessPro 支配」需要按变体挑不同模式：TB2 的 Medium（76.3 / 0.56）支配 dynamic routing / budget hooks（76.3 / 0.92），但 low reasoning（62.7 / 0.45）更便宜、需 Lite（72.9 / 0.43）支配；不存在单一模式支配全部三个变体（Table 3，p.9）。
5. **生成测试的质量**：正文只强调 recall（41.7%→83.3%）；Figure 2 有 1/8 假阳性，但正文未给 precision；固定测试可能被候选适配，held-out test 只能缓解（Figure 2，p.6；§4.3）。
6. **3/2 次运行的语义不清**：§5.1 说 TB2/SWE 的 mean±std 来自 3 次、PB 来自 2 次；Figure 4 题注说最终点是 "means across repeated test runs"。是独立重新演化还是同一批冻结 harness 重复测试，原文未说明；std 的解释因此不同（§5.1，p.7；Figure 4，p.9）。
7. **选择规则的边界**：归一化用的是当前候选池的极值，换候选池会改变同一 harness 的 J_m；若 P_max=P_min 或 C_max=C_min，分母为 0；k=10、G=3、epoch 数、每轮最多保留 3 个、权重 (1,2)/(1,1)/(2,1) 都没有消融（§4.3、§A.2）。
8. **覆盖与复现**：只有 2 个 backbone × 3 个 benchmark；AFlow/SkillMOO/AHE 只在 Table 1 做定位、没有同等实验；代码仓库只核到结构/README（MIT，阅读时 ★84），未端到端跑通；README 的 "Paper (coming soon)" 徽章与 arXiv v2 不符（§2、Table 1；§5.1）。

版本关系

- **v1 → v2**：v1 2026-10-19、v2 2026-10-20，均 41 页；差异是第 2 单位 `Z.AI Co., Ltd.` → `Z.AI`、第 5 单位 `Kuaishou-Kuaishou Technology` → `Kling AI`、补共同一作/通讯作者标记；正文结构与 Table 2 数字不变。
- **v2 PDF 的两次抓取**：2026-10-25 使用的副本（sha `ce88e54a…`；生成方式/与当前 v2 的差异原因未确认）p4 Figure 1 不完整；2026-10-26 从同一 URL 重取的当前 v2（sha `c633082a…`）Figure 1 完整；两版除 p4 外逐页文本一致。本文按当前 v2 写；旧副本的 Figure 1 缺陷不算当前原文的问题。
- **代码/数据**：代码仓库 `pzs19/HarnessPro`（MIT；创建 2026-09-24、最近 push 2026-10-19、阅读时 ★84）；顶层含 `harnesses/{terminal-bench-2,swe-bench-pro,programbench}/{lite,medium,heavy}`、`scripts/`、`splits/`、`src/`、`tests/`；本次核对未端到端复现。无 v3。

00214 Dream-RSI：用「发现历史」当模拟器来改进探索策略

> 版本：arXiv v2（43 页，2026-10-06；v1 2026-09-14）。**注意**：本文采用的 v2 与项目页版/arXiv v1（36 页，仓库归档 `papers/00214-Dream-RSI.pdf`）在目标函数与若干口径上不同，见本节末尾〈版本关系〉。

一句话

把每次搜索留下的**发现树**当成一台零执行成本的「模拟器」：树里每个节点都带着那次尝试**真实发生过**的评测结果，于是要评价一个新探索策略时不必重跑 rollout，只要在这棵（以及所有历史）树上**回放**——只把已记录的结果揭示给策略看，让策略做决策，再由回放评分给这份决策表现打分。用它做的是**元层面**的自我改进：被进化的不是解，而是"怎么搜"。

它要解决什么

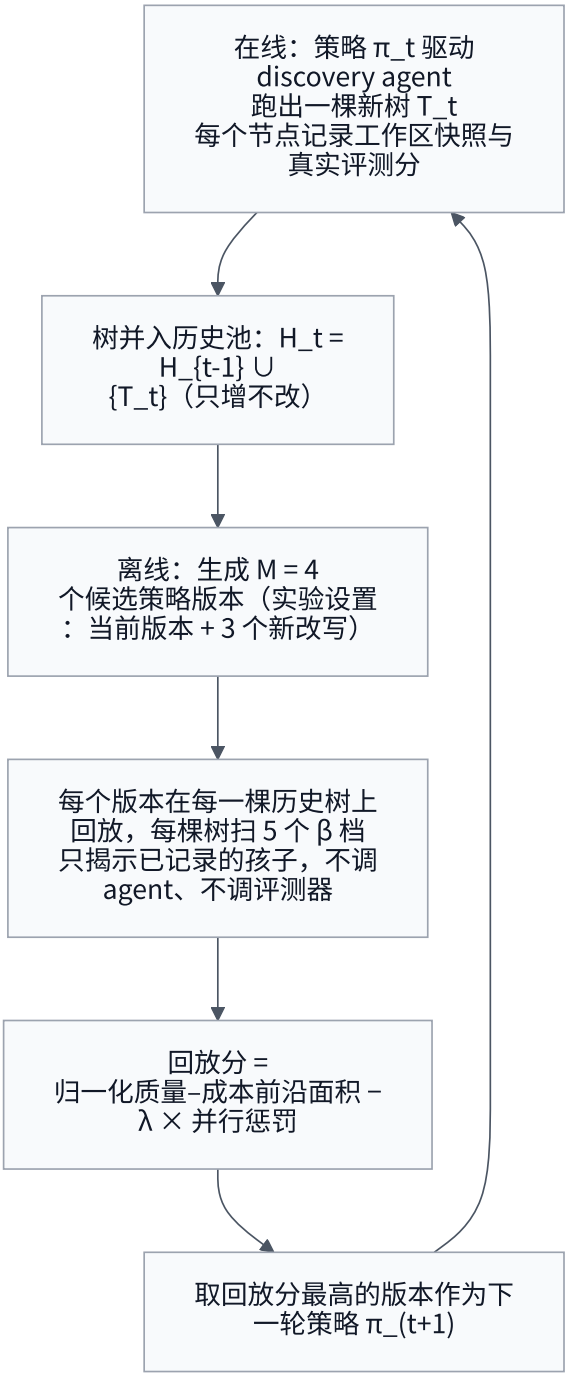
- 现有系统用**人工写死、全程不变**的探索策略（先开哪些方向、每步并行多少、何时剪枝停止），无法从积累的经验里学习，会反复把算力花在无效方向上（v2 §1，p.1）。
- 直接在线上优化探索策略有**两个瓶颈**（v2 §1，p.1）：① 元级反馈**延迟且昂贵**——评价一个候选解只需一次评估，评价一个**探索策略**要看它如何塑造几十上百轮"提议—评估"的走向；② **元策略空间巨大**，要试很多个才能碰到好的。
- 与已有「复用历史」工作的差别（v2 §1 p.2、§6 p.11）：别人把历史当**静态上下文**或**微调数据**；Dream-RSI 把历史组织成**回放模拟器**，让"评价探索策略"这件事变便宜。它也不像 Dreamer 系那样**学**一个动力学模型——用的是"已探索过的发现空间"这份**有据可查的记录**（v2 §2，p.4）。

系统由哪些部件组成

部件	具体是什么	作用	输入 → 输出
发现树（discovery tree）	根 = 初始工作区；每个非根节点有唯一父节点，附带**工作区文件系统快照、产物、评测诊断、分数**（v2 §3，p.4）	记录"探索到过哪里、结果如何"；它同时也是回放模拟器的载体	一次尝试 → 一个节点（子节点）
discovery agent（探索 agent）	被策略驱动的执行者（Lasso/数学/GPU kernel 用 Gemini 3.1 Pro，其中 Lasso 另用 Gemini 3.7 Flash 跑一遍；harness 域用 Claude Opus 4.8 + Claude Code；v2 §4 与表 3）	在指定节点的工作区里生成一个新候选并交给评测器	节点工作区 → 新子节点
评测器（evaluator）	每个任务固定的评分协议（kernel 任务查数值正确性；Lasso 以正确性为**门槛**、以运行时间为分数）	给节点打分——**这些分是回放的全部素材**	产物 → 分数
探索策略 policy（π）	一份 Python 代码：类 `OptimalPolicy`，含 `solve(question, budget)` 与 `plan_grid(context)` 两个方法（v2 附录 D.2，p.23 起）	决定"从哪继续、并发多少、何时剪枝停止"；**这是被进化的对象**	观测（已揭示节点/合法动作/基线分/最大并行） → 下一批要探索的节点
历史池 H_t	历次在线 rollout 产出的树集合，只增不改	离线阶段的世界集合	树集 → 回放环境
policy-development agent	读回放轨迹、分数与上几轮反馈、改写策略代码的 LLM（模型归属：harness 域明确为 Opus 4.8；其余域原文未注明）	改写策略代码，生成新版本	回放反馈 → 新策略版本
β（预算偏好旋钮）	策略内部的单一标量：高 β = 更宽、更有耐心、更弱剪枝（v2 附录 D.2）	在 0 到 1 之间调"探索 vs 收敛"	—

一次迭代怎么走

外层第 t 轮 = **一次在线 rollout** + **一段离线筛选**（v2 §3，pp.4–6）：



关键机制说明：

- **在线阶段的决策与真实执行**：每轮决策选一个可行批次（最多 W 个节点，W 即并行 worker 数），每个被选节点恢复其工作区、生成一个候选、评测、挂成一个子节点；批次为空即提前结束（分叉后**根始终可选**，用于开新分支）。预算用尽或跑满 K1 轮也会结束（K1 无数值）。树的最终形态记为 T_t。
- **回放的"模拟"是怎么回事**：回放里对节点 v 的"继续"不再真实执行，而是**揭示记录里 v 的唯一子节点**（若在本次回放中尚未揭示）；对根则揭示"**最早创建的、尚未揭示的那个分支**"——即每次探根只开启一**个**没看过的分支（v2 §3，p.5）。策略只能看到已揭示的子树和合法动作信息，**看不到未揭示节点的结果**（写在策略开发提示词里的硬约束）。
- **回放是有边界的**：请求超出冻结树已有规模（分支数、每分支 refine 数）的网格属于「out of support」，**拿不到回放奖励**（v2 附录 D.2 的策略改进提示词，Listing 2 跨 pp.23–27，该句在 p.26）。
- **离线规模**（表 3 的实验设置）：M = 4 个版本 × 每棵历史树 × 5 个 β 档（0 / 0.25 / 0.5 / 0.75 / 1）；每个版本在每个 (树, β) 组合下从根**重新开始**回放，不继承上一次的已揭示子树。("零执行成本"指回放不调 agent、不调评测器；回放本身的算力不在成本口径内，见〈陷阱与边界〉。)

轮与轮之间更新了什么

更新什么	载体	谁写	何时写	可否回滚
探索策略代码	`OptimalPolicy` 所在的 Python 文件	policy-development agent	每个外层轮的离线阶段（每轮新增 3 个版本）	可：历史版本留存，选择是取回放分最高者且**当前版本也在候选集里**
发现树	节点的工作区快照 + 产物 + 诊断 + 分数	在线阶段由 discovery agent 与评测器产生、编排层落盘	在线 rollout 每轮之后	不可：记录树「永不改变」，历史池只增
β 的默认值	写进策略代码的配置默认值	policy-development agent（按最近 2–3 轮线上 cycle 的证据小步调整，步长约 0.1–0.2）	提出新策略版本时一次性确定	可（随策略版本一起换）
下一轮网格规模（分支数 × 每分支尝试数）	`plan_grid` 的返回值	策略代码（确定性方法，不得窥探本 episode 的结果）	每个新在线网格创建之前	可（每轮重规划；受硬上限与记录规模约束）
不动的东西	base agent 模型、评测器、执行接口	—	全程不变	—

什么时候停、什么算「更好」

- **回放分的构成**（v2 式 (1)，p.5；与 v1 不同，见〈版本关系〉）：先在 5 个 β 档下各回放一遍，得到 5 个「花的调用数 vs 归一化质量」点；取其中的非支配前沿，算**前沿围出的面积**（奖励"以更少调用换更高质量"），再减去**并行惩罚**（平均"每次调用占用的串行轮数"——逼策略把独立探针成批发出去而不是逐个串行跑）：

$$V_i^m := \frac{1}{B_i}\sum_{j=1}^{L_i^m}\left(\bar{a}_{i,j}^m - \bar{a}_{i,j-1}^m\right)\left(B_i - \bar{n}_{i,j}^m\right) - \lambda\,\frac{1}{|\mathcal{R}|}\sum_{\substack{\rho\in\mathcal{R}\\ n_{i,\rho}^m>0}}\frac{k_{i,\rho}^{m,\star}}{n_{i,\rho}^m},$$

$$\mathcal{R} := \{0,\ 0.25,\ 0.5,\ 0.75,\ 1\},\qquad \bar{a}_{i,0}^m := 0$$

  读法：第一项是该次回放把"质量–成本前沿"撑开的面积（质量按该世界历史最好分归一、调用数按该世界全部已评测节点数归一；起点 $\bar{a}_{i,0}$ 约定为 0）；第二项是并行惩罚，λ 控制强度。跨树平均后，取回放分最高的版本作为下一轮策略。

- **唯一的理论保证**（v2 §3，p.6）：因为候选集里包含当前策略本身，所以下一轮策略在**固定历史 H_t 上的平均回放分**不会比当前差。**这只对回放这个代理指标成立**，对真实在线搜索质量没有任何保证。
- **停止**：外层轮数是 5（Lasso）/ 9（kernel 与 harness 域）/ 10（数学域）（v2 表 3，p.17）；内层由策略自己"选空批"终止（在线最多 K1 个决策轮、回放最多 K2 个；回放在"整棵冻结树已揭示完"时也结束；K1/K2 全篇无数值）。

监督信号从哪来

- 节点分数来自**真实执行**（固定评测协议），回放不产生新分数、只揭示旧分数；回放分是"对已记录结果的重放统计"。
- 没有 reward model、没有外部标注：模板是"搜索——记录——回放筛选"。（注意：回放目标在代码里叫 `pareto.reward`，但那是重放统计的名字，不是奖励模型。）
- policy-development agent 读：各版本的回放轨迹与分数、历史轮目录里的策略代码与扫描结果、每轮归档的"每个 (冻结树, β) 一条回放 episode"的执行轨迹、最近 2–3 轮线上 cycle 的清单文件。写：只写策略代码文件（附一段说明前缀信号、批规则、β 调度、网格规划与护栏的 docstring）。
- 有明确的**污染护栏**：执行轨迹只能作为轮间反馈，不得在 `solve()` 内读取，也不得把 trace 里的具体分支号、cell id、分数拷进策略逻辑。

关键数字

事实	数字	出处
外层轮数与每轮预算	Lasso：5 轮 × 110 次调用（10 个分支 × 每分支 11 次尝试＝首次探索 1 次 + refine 10 次）；Lasso 的 Flash 骨干版为 32 × 20 = 640；kernel 9 轮、数学 10 轮（同为 110 次调用/轮）；harness 域 9 轮 × 16（4 × 4）	v2 表 3，p.17
Lasso 时间与调用（相对固定策略）	Pro：3587.1 → 2931.0 ms，调用 550 → 317；Flash：2516.7 → 2350.6 ms，3200 → 1879	v2 §4.3，p.8–9
对比 SimpleTES	调用少 162× / 27×（但**骨干模型不同**，v2 自注 caveat）	v2 §4.3，p.9
RCV1 警告	Pro 的均值改善**由 RCV1 单集驱动**，其余五个数据集反而变慢（v2 新增 caveat）	v2 §4.3，p.9

事实	数字	出处
数学三个任务	Sum-Difference 1.145427（高于自己的固定基线、也高于 SimpleTES）；Autocorrelation 1.456375（**差于**自己的固定基线 1.456001，也差于 SimpleTES 1.453675）；Circle Packing 2.635983（并列最好）	v2 表 1，p.7
GPU kernel	VGG16 / LayerNorm：达到相当性能所需**代数**少 2.43× / 1.79×；ConvDiv / ConvMax：相当预算下性能高 2.09× / 1.44×（后者由最后一个点的单点尖峰贡献——图内读数，见笔记的像素测量）	v2 §4.1 与图注，p.6–7
harness 域（v2 新增）	种子 harness（Terminus-Kira）best-so-far 0.7247 → 0.8090（+8.4 个百分点）；对照柱：Claude Code 0.75、Terminus-Kira 0.72、Meta-Harness 0.79（0.72 是对照柱读数，与种子起点 0.7247 的细微差异原文未解释）	v2 §4.4 与图 5，p.9
PACEvolve 基线（v2 新增）	48 小时上限下的最好分（图 3 内标签）：VGG16 0.2002、LayerNorm 0.8264、ConvDiv 0.6452、ConvMax 0.2654；附录 C 另给 48 小时预算的完整曲线。**看图注意**：图 3 中该基线的橙线位置与标签数值不一致（疑为原文绘图问题），引用以标签为准	v2 图 3，p.6–7；附录 C，p.21
多积累历史的好处	ConvDiv 上"全部历史" 1.90 分 / 786 次调用 vs "仅当前历史" 1.59 / 430	v2 表 2，p.10
历史当提示引导（负结果）	ConvDiv 终值：Dream-RSI ≈1.89、加引导 ≈1.44、固定策略 ≈1.28、固定+引导 ≈0.64	v2 §5 与图 6，p.10
探索行为的演化	ConvDiv 9 轮的轮内最好分 0.427 → … → 1.898；评测尝试数 110 → … → 86（先省、后回升）	v2 §5 与图 7，p.10

陷阱与边界

1. **回放只在"已经走过的格子"上有定义**：策略提出一个记录里没有的新方向，回放给不出奖励——所以改进循环**天然偏向加深已探索的分支**，对"本该开新根"的策略没有激励。论文的消融只测了"历史世界数量"，没测这一条。
2. **保证只是代理指标上的**：回放分不降 ≠ 在线结果更好；论文没有回放分与在线表现的相关性分析；图 4(b) 里 Dream-RSI 自己的 Pro 曲线第 4 轮就出现过回退（固定策略曲线反而单调下降）。
3. **成本口径不完整**：成本记的是 discovery-agent 调用数，**不含** policy-development agent 的 LLM 调用、每轮"版本数 × 历史树数 × β 档数"的回放算力、以及快照存储。随外层轮数增长，离线评估量线性增长。
4. **关键超参未公开**：K1、K2、λ 在 v2 全篇与 4 份代码清单中均无数值；数学主表未注明用哪个任务变体（Autocorrelation 的 Φ 变体、Circle Packing 的 n）。
5. **统计强度弱**：ConvMax 的 1.44× 由单点尖峰贡献；Lasso 均值由 RCV1 主导；无多种子/方差；harness 域**没有固定策略对照**（论文估计再跑一条完整搜索要超过 10 万美元）。
6. **自评风险**：harness 域里 Opus 4.8 同时充当 discovery agent、policy-development agent 与评测模型；且代码未发布（2026-10-15 经仓库 API 核对：仍只有 README/CITATION 等文件，无代码、无 license）。
7. **前缀可见性靠"不许"而非"做不到"**：策略代码与 trace 同在运行环境里，越权读取由提示词禁令约束，回放评分的可信度依赖 agent 守规矩。

版本关系（v2 vs 项目页版 / arXiv v1）

- **目标函数被重写**：项目页版与 v1 的式 (1) 是三分项——"最好分 − β1 × 工作量 + β2 × 并行奖励"，与附录里实现的 `pareto.reward = pareto.auc − λ × parallel_penalty` **不是同一个目标**；v2 把正文式 (1) 改写成与实现同构的「前沿面积 − 并行惩罚」，并补上 β 栅格与 M=4。引用方法时按 v2。
- **口径收紧（v1 的夸口被删）**：数学域 "matches or surpasses … within 1k generations / 50× savings" → v2 "matches or exceeds … on **two of three tasks**"；kernel 域 "identical budget" → "**a comparable budget**"；"consistently achieves superior downstream performance" → "**By the final round** …"；并补上 RCV1 与跨模型骨干两条 caveat。
- **内容扩充**：v2 从 8 任务 / 3 域扩为 **9 任务 / 4 域**，新增 harness optimization 一节与 PACEvolve 基线；两版共享的数字（Lasso 表、数学表、kernel 四个乘数、E0–E8 序列、附录两份提示词）逐项一致。

00212 RRSI：给 harness 演进的「搜索轨迹」加正则

> 版本：arXiv v3（27 页，2026-10-04；v1 2026-09-21、v2 2026-09-23；论文未说明版本间改动，本文与笔记均基于 v3）。

一句话

"用 agent 自己跑出来的反馈反复改 harness"这套循环的失败模式被诊断为**过拟合**：候选的提出依赖前几轮在**同一批任务**上的测量，测出来的涨分越来越像自欺。RRSI 不限制"能改什么"（harness 的 9 类组件全部开放），只在**提议**与**选择**两侧给**搜索轨迹**加约束——限制"一轮能动多少、动哪儿"和"哪些测得的涨分有资格变成永久状态"。

它要解决什么

- 标准循环（论文式 2，p.3）：第 t 轮把当前 harness 在 evolve set 上跑出轨迹 → 汇总反馈 → proposer 生成候选 → **在同一批 evolve set 上评估** → 取分最高者当下一轮。
- 关键风险：对 evolve set 的复用是**自适应**的（第 t 轮的候选取决于前几轮在同一批任务上的测量），相当于"在极其表达力强的搜索空间上做自适应经验优化"——反复拿同一份数据做选择，测量本身会失真（§2，p.3）。
- 三种过拟合行为（§1，p.2）：**benchmark-specific fitting**（把评测集特有内容——任务名、实体名、任务特有取值、答案——写进 harness）、**noise chasing**（在有限样本的噪声里挑"赢家"，把随机波动固化成永久状态）、**complexity accumulation**（堆上下文与推理成本换 evolve 分）。
- 论文给的证据（图 1a、表 1）：Meta-Harness 在 Harvey LAB 的 evolve 集上 93.0（高于 RRSI 的 90.5），但 OOD 平均只有 40.6；AHE 与 TTHE 的 OOD 平均甚至**低于未演化的初始 harness**（39.2 / 38.0 vs 39.7）。

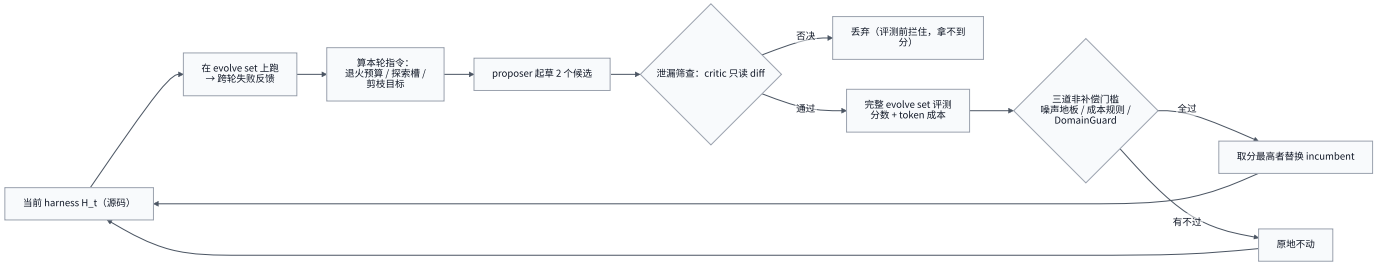
系统由哪些部件组成

部件	具体是什么	作用	输入 → 输出
policy（被冻结的 LLM）	主实验全程 Claude Opus 4.8，不训练、不更新（另有换 policy 与弱 backbone 的迁移实验，见「关键数字」）	执行任务；它跑的轨迹是唯一反馈来源	任务 → 轨迹
analyst	同为 Opus 4.8	在 evolve set 上跑当前 harness、汇总**跨轮失败反馈**	harness + evolve set → 失败反馈
proposer	同为 Opus 4.8	从约 80 条编辑的池子里起草候选（每轮 2 个，每个候选是池的一个子集）	全历史台账 + 本轮指令 → 候选 diff
leakage critic	同为 Opus 4.8	**评测前**读候选 diff，拦下把评测集特有内容写进代码的候选	候选 diff → 通过 / 否决
selector	算法（非 LLM）	应用三道非补偿准入条件，决定谁替换 incumbent（泄漏筛查在评测前由 critic 执行）	分数/成本 → 下一个 harness
编辑台账	每条**被测量过**的编辑一条记录：轮次、组件、假设、diff、分数变化、成本变化、是否当轮获胜	让后续提议条件于全历史；被否的机制作为**负证据**保留	—
9 类组件词表	`prompt / control_flow / config / output_plumbing / context_mgmt / client_tool / skill / memory / subagent`	编辑必须打组件标签，用于剪枝与探索统计	—
噪声带 δ	开跑前反复评测**未改动**的 harness 估出的经验噪声带	三道准入条件里的"地板"	—
DomainGuard	每个领域自定的额外验收条件（论文未展开，全文最大黑箱）	与地板、成本规则并列的第三项合取条件	—

一次迭代怎么走

一轮 = 6 步（按 Algorithm 1/2 整理；超参与运行细节见 §4.1）：

```
1. 分析    analyst 跑当前 harness H_t，汇总跨轮失败反馈 F_t
2. 拟题    计算本轮指令：
             • 退火编辑预算 b_t（式 4：早期允许多改，后期只留 1 处可归因改动）
             • 停滞标志与"从未被测量过的组件"探索槽（式 13）
             • 剪枝目标 B_t（式 11/14：近期窗口内没有正贡献的组件，交给 proposer 删；
               窗口按 coding/agentic/eng 依次为 4/4/5 轮）
           proposer 从约 80 条编辑池起草 m = 2 个候选，每条编辑带组件标签与假设
3. 筛      泄漏 critic 只读 diff（不读候选的说明文字）：
           拒绝显式编码任务名/实体名/任务特有取值/答案，或"惰性机械"的候选
           —— 在评测之前执行，让泄漏候选拿不到虚高的 evolve 分
4. 评      通过的候选在完整 evolve set 上并行评测（每任务 k 次 trial）→ 分数与成本
5. 准入门  三道非补偿条件（另有评测前的泄漏筛查，见步骤 3）：
           E 噪声地板：候选分 ≥ 历史最佳分 − δ
           F 成本规则：增益超过 δ 时，要求成本涨幅 ≤ 基础容忍 + 每分增益额度；
                       增益落在带内时，改为要求"省 token 或首次触碰 4 类结构性组件"
           G DomainGuard：领域自定的验收条件
           —— 三者是合取（非补偿性）：任何一项不过，别的项再漂亮也不能准入
6. 选择    可准入者中取分最高者作为 H_(t+1)；无合格者则原地不动
           更新"历史最佳分"（只升不降）并把每个被测编辑连同胜负记入台账
```



轮与轮之间更新了什么

更新什么	载体	谁写	何时写	可否回滚
当前 harness	源码文件（diff 形式演进）	proposer 生成候选，selector 决定替换	每轮评测与门槛之后	论文**没有定义显式回滚**；唯一"删掉已接受内容"的通路是结构剪枝（列入删除目标后仍要由 proposer 提议删除、再过门槛）
编辑台账	历史记录（按编辑逐条记）	系统记录	每轮评测后	只增不改
历史最佳分	标量，只升不降	selector	每轮选择后	单调不减（"地板"只会随时间收紧）
每轮指令（预算/探索槽/剪枝目标）	当轮参数	公式计算	每轮开始	当轮有效
噪声带 δ	标量	开跑前一次性标定	run 之前	作者明确它**不许调**（调低等于用噪声买分）
不动的东西	policy、judge、工具环境、评测窗口	—	全程不变	—

什么时候停、什么算「更好」

- **提议侧三个正则**（§3.2）：
  - **退火编辑预算**：一轮更新里可**独立归因**的编辑数上限，从首轮的多改（coding 4 / agentic 3 / eng 4）按余弦退火降到末轮的 1。它约束的是"一次捆绑多少条编辑"，**不是** harness 能包含哪些组件：

$$b_t = \left\lceil b_{\min} + (b_{\max} - b_{\min}) \cdot \tfrac{1}{2}\left(1 + \cos(\pi t / T)\right) \right\rceil$$

    读法：前期允许多个协同改动一起上（便于发现新机制），后期只留单一可归因的改动，让"哪个改动带来涨分"说得清。
  - **证据化信用分配**：每个被测量过的编辑都进台账（含失败者），后续 proposer 条件于全历史。
  - **结构化探索**：w = 3 轮内进展不超过噪声带即判定"停滞"，停滞时预留 1 个候选槽位（共 2 个）给**从未被测量编辑过**的组件。
- **选择侧门槛的精确定义**：
  - 泄漏筛查在**评测前**执行，且 critic 读的是 **diff 而非候选的自我说明**——候选可以在说明里点名它想修哪个任务，但那些内容不能出现在 diff 里。
  - 噪声带 δ 由重复评测未改动 harness 得到：coding 0.017（≈178 次 trial 里的 3 次通过）、agentic 0.004（≈14,100 条 rubric 判定里的 60 条）、eng 0.020（≈244 次 trial 里的 5 次）。
  - 成本规则里的 β1 是把"评测单位的增益"翻译成"允许的成本增长"（coding 的 β0=0.10、β1=44.5，178 次 trial 下折合每次通过 25%）：coding 上"多 7 次通过"允许成本增长 10 + 7 × 25 = 185%（即成本最多涨到 2.85 倍；实测 130% 涨幅被接受），"多 1 次通过"只允许 35%（同样的成本被拒）。
  - 带内规则只给 4 类**结构性组件**（client_tool / skill / memory / subagent）发"新颖性加分"；coding 域的分数权重取 0，即带内涨分本身不算信用。
  - 所有超参**开跑前固定、无 sweep、不看任何留出/OOD 集**。
- **停止**：20（coding）/ 20（agentic）/ 40（eng）轮；无候选合格就原地不动。

监督信号从哪来

- 全部来自各 benchmark 的判分：Terminal-Bench 2.1 与 EngDesign、Frontier-Eng 是**确定性判分**（任务自带单测/冻结模拟器）；Harvey LAB、JobBench、APEX-Agents、GDPval 是 **LLM judge**（Harvey 与 APEX 用 Gemini 3.5 Flash 逐条判定，JobBench 取双 judge 均分，GDPval 用三模型评审团算对人工专家的胜率）。

- 论文声明所有演化臂与 baseline 在**同一时间窗口、同工具环境、同 judge、同 trial 数**下评测；APEX 分母固定为全部 480 个任务（基建失败记失败，防止"崩在难任务上反而好看"）。

关键数字

事实	数字	出处
agentic workspace 逐方法对照	初始 harness 五列依次为（Harvey evolve / Harvey 同分布留出 / JobBench / GDPval / APEX）：89.4 / 86.9 / 36.0 / 48.8 / 34.2；Meta-Harness evolve 最高 93.0 但 OOD 平均 40.6；**RRSI evolve 只涨 1.1 个百分点（最低）**，OOD 三项全列最高（40.7 / 52.3 / 37.9）	表 1，p.8
相对初始 harness 的涨分	evolve：TB2.1 +6.0、Harvey +1.1、EngDesign +4.9；留出：SWE +1.8、Harvey 同分布 +2.3、JobBench **+4.7**、GDPval +3.5、APEX +3.7、Frontier-Eng +4.3（六个留出 split 无一退步；上述 Δ 均为百分点）	表 6，p.24
成本	2.42M token/trial vs 无正则演化 3.80M（≈ −36%；摘要写 30%，与表格不符）；初始 harness 1.56M；AHE 最贵 3.82M（+58%）、OOD 低 4.4 分	表 2，p.9 / §4.3
消融（agentic）	无正则：evolve 92.8 / OOD 40.3 / 3.80M；去接受侧：91.5 / 41.0 / 3.59M；去提议侧：90.7 / 41.9 / 2.69M；完整：90.5 / 43.6 / 2.42M——两组正则**各自**贡献迁移能力	表 2，p.9
跨 policy / 跨模型	Gemini 3.5 Flash 当 policy：TB2.1 64.6 → 78.7（+14.1 个百分点）；演化出的 harness 直接跑更弱的 Gemini 3.1 Flash Lite：11.2 → 14.6（相对 +30.4%）	表 3/4，p.9–10
复跑验证	每个演化 run 整体重跑一遍（只重抽 proposer 随机性）：9 个 split 两次都不退步，OOD 平均 43.6 vs 43.7，最大分歧 1.8 分（TB2.1）	表 7，p.25
定性对照（20 轮 32 个候选中只接受 4 处编辑）	4 处全接在基础 harness 已有的"提交前回弹"上：一次覆盖度复查 + 三个标准库探针（把交付物读回来指出缺什么）——探针报告**结构性事实**，不带 rubric 词汇，故在 OOD 上仍触发；Meta-Harness 的 730 词工作法复述了 evolve suite 的评分体系，TTHE 的约 8,500 词协议块是 evolve 特有的交付物体裁	表 9，p.27

陷阱与边界

1. **"更省 token"一部分是设计结果**：成本规则**直接拒绝**贵候选——把它读成"正则有免费收益"要小心。
2. **演化本身是花钱买分的**：未演化的初始 harness 是 1.56M token / 21.2 steps，比演化后（2.42M / 26.3）更省；RRSI 决定的是"花多少"。
3. **摘要与表格不一致（引用用表格值）**：摘要"30% fewer tokens"与表 2 的 3.80M → 2.42M（≈36%）不符；摘要"up to 4.3 points on OOD"与表 1/表 6 的 JobBench +4.7 不符（4.3 只对 Frontier-Eng 成立）；引言"up to 12.5%"只在 agentic 三项范围内是最大值（按四个 baseline 均值**复算**，全留出最大为 Frontier-Eng 的 22.9%；原文正文另印有**相对初始 harness** 的 24.3%——两种相对口径并存，引用时注明分母）。
4. **§4.2 正文"consistently outperforms baselines on all held-out datasets"有反例**：Harvey LAB 同分布留出上 RRSI 与 Meta-Harness **并列 89.2**。
5. **权重取值缺失**：带内规则的三个权重论文称"在表 5"，但表 5 没有这三行，全文只给定性说明（coding 的分数权重为 0）。
6. **消融只在一个领域**（agentic）；**proposer/analyst/critic 与 policy 同源**（全是 Opus 4.8），换弱 proposer 是否成立未测；δ 与评测规模强耦合，任务量小或 judge 方差大时估计本身不稳。
7. **"L0/L1/L2 正则"只是功能类比**：不是优化带范数惩罚的目标，也不把异构组件当作连续参数向量的坐标（§3.1 与附录 D 开头反复强调）。正文里还有一处把成本规则写成"L1-style"（p.9），与 §3.1 给成本准入的 Ridge/L2 定位对不上，疑为笔误。
8. **DomainGuard 是黑箱**：论文只有一句"每个领域自定的检查"，具体检查什么全文未展开。

00205 Prime Agent：把执行证据变成持久的 harness 状态

> 版本：arXiv v1（16 页，唯一版本；2026-08-05 另有一版博客，差异见本节末尾〈版本关系〉）。

一句话

论文对「自我改进」的定义非常克制（§2.5，p.5，原文）：*Self-improvement converts execution evidence into persistent harness state that changes later behavior while model weights remain fixed.*——把**执行证据**转换成**持久的 harness 状态**，从而改变后续行为；**模型权重全程不动**。整个系统（Prime Agent）是围绕这一点的工程实现：让长时程评测里的"harness 失败"（丢状态、限制动作、算错资源、提前终止）不再变成"模型失败"。

它要解决什么

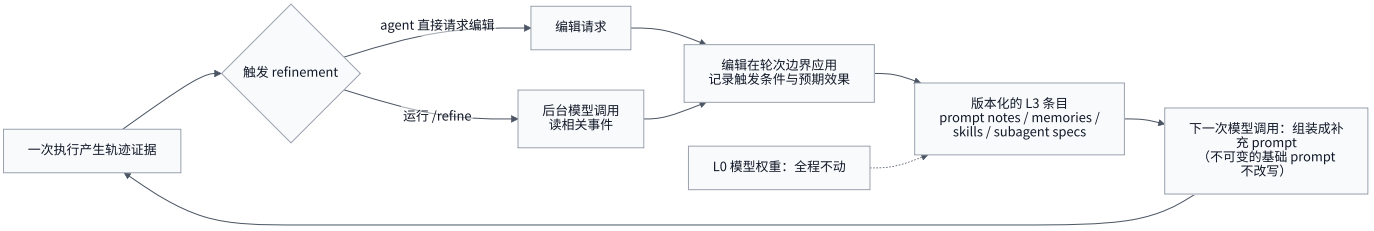
- harness 是模型观察与作用于世界的"膜"：模型应该因为任务超出能力而失败，**不应该因为 harness 掉状态、限制有用动作、算错资源、提前终止而失败**（§1，p.2）。
- 评测动机：长时程最流行的两类指标（固定支出下的分数、实用平台期分数）在 harness 不可靠时，测的是 harness 而不是模型（§1，p.2）。
- 由此定出 harness 的关键属性：**表达力**——不编码某一条工作流，而是暴露原语，让模型在推理时自己构造程序、子 agent 与反馈回环（§1，p.2）。

系统由哪些部件组成

部件	具体是什么	作用	输入 → 输出
持久 IPython REPL	每个会话一个持久 REPL；中间值跨轮存活、**在被选中前不进上下文**；工具以 Python 模块导入	避免反复把大日志、任务规约、结构化评测输出序列化进上下文；测试时算力拆成"模型推理 / Python 执行 / 工具调用"三块分别记账	模型写的 Python → 保留值（L2）或产物（L3）
`rlm()` 原语	异步原语：**调用它 = 创建并调度一个子 agent 会话，并在子 agent 完成之前返回一个稳定句柄**；结果稍后经 agent 间直连消息到达；句柄在压缩或重启后仍可用来追问	让"递归委派"成为程序化计算的一部分：父会话继续本地计算，不必阻塞等答案	任务文本 + 名字 → 句柄（不是答案）
递归子会话	子 agent 有自己的模型上下文、IPython kernel、历史与工作区；daemon 持有会话，agent 之间可寻址父/子/兄弟；消息走异步队列	并行分工；会话拓扑在卸载/断连后保留	消息 → 目标会话队列 → 恢复后消费
Continual Harness（四类类型化状态）	① prompt notes = 行为指令；② memories = 事实；③ skills = **可执行过程**；④ subagent specs = 可复用角色/分工。条目可增删改查；**默认会话本地**，显式请求的全局条目才对后续会话可见	这是被"自我改进"改写的那部分状态	轨迹证据 →（refinement）→ 版本化条目 → 后续调用的补充 prompt
daemon 与生命周期	会话生命周期 ADMITTED → RUNNING → IDLE（已加载但无活动轮次）→ INACTIVE（已卸载但可从持久状态恢复）；客户端断开不停会话	长时程会话可恢复、可重连	—
Agents View	给人用的界面：查看历史、attach、注入输入、detach 而不打断执行；`agent-observe` 只读状态、`agent-message` 定向消息	人机协同	—

一次迭代怎么走

这里的"迭代"不是训练轮，而是**证据 → 状态**的循环：



- **refinement 的完整流程**（§2.5，p.5）：agent 直接请求编辑，或运行 `/refine`——它在相关事件上开一次**后台模型调用**；每次编辑**在轮次边界应用**，记录触发条件与预期效果；版本保留来源并**支持回滚**；refinement **只补充不可变的基础 prompt，不改写基础策略**。
- **证据 → 状态的映射规则**（§2.5）：有用的计算 → skill；重复出现的协调模式 → subagent spec；被修正的假设 → memory 或 prompt note。
- **四级状态与各自的"写机制"**（图 2，p.4）：

层级	内容	写机制（原文用词）
L0	模型权重	fine-tuning（本文**不用**）
L1	单次模型调用的 token 可见工作状态	compaction（压缩：把对话前缀换成摘要，**原始事件留在 L3** 供 REPL 取回）
L2	代码、工具、保留的值、递归会话状态	agentic garbage collection（模型随任务变化自己创建、保留、摘要或删除 REPL 值与子会话）
L3	历史、产物、记忆、技能、prompt、subagent 规格	refinement（版本化被选中的条目）

跨层移动全靠**显式操作**：L2 的值被序列化进 L1 才进入生成；L3 条目被运行时组装进后续补充 prompt。

轮与轮之间更新了什么

更新什么	载体	谁写	何时写	可否回滚
L0 权重	模型参数	—	**全程不动**	不适用
L1 活动上下文	token 序列	运行时（compaction）	压缩时	前缀被摘要替换，但原始事件保留在 L3 可取回
L2 值与会话	kernel 状态、保留值、子会话	**模型自己**	随任务变化	论文未提
L3 Continual Harness 条目	版本化条目（四类）	refinement（直接请求或 `/refine`）	**轮次边界**	**可回滚**（版本保留来源）
事件历史	append-only 历史	运行时	持续	不删（分叉也不删旧序列）

什么时候停、什么算「更好」

- 长时程执行的三种控制机制（§2.6，p.6）：**autonomous mode**（在显式预算内持续跑轮次；**每轮之后**跑任务指定的结束条件测试，失败则返回有界输出再试；轮数/token/墙钟上限停止）、**goal**（目标跨续跑保留，由 agent 自己标记完成结束）、**heartbeats**（cron/定时触发轮次）。
- 评测配置把任务与工具接口、模型与 provider 设置、压缩与 refinement 策略、重试策略、完成门与资源上限**绑成一个对象**；**成本核算聚合 root 与全部后代会话**（委托出去的工作不会成为成本盲区）。
- 没有量化"改到什么程度算好"的门槛——改进是否有效由后续任务成绩体现。

监督信号从哪来

- **没有 verifier 模型，也没有 reward**：全文 0 处 "reward"；"verifier" 共 4 处——正文 3 处（事件历史里的 verifier outcomes、EmulatorBench 的诊断测试、图 7 图注）指的都是**任务侧**的外部检查，不是通用奖励模型；另 1 处在参考文献标题里。
- refinement 的"反馈"就是**轨迹证据本身**：由那次后台模型调用判断哪些事件相关、哪些编辑有帮助。论文未定义"哪些事件算相关"，也没有对 refinement 的质量做评估。

关键数字

事实	数字	出处
ARC-AGI-3	Prime Agent + Opus 5 的 RHAE Best@1 达 95.5%；**"30% → 95.5%" 的两端不是同一实验的对照**：30% 是外部参照点（Opus 5 跑 ARC 官方 harness 的 30.2%），论文明确参照值"只用于定位，不构成因果隔离的 harness 效果"	摘要 p.1、图 5 图例 p.7、§3.1 p.7
长上下文套件	9 任务 × 3 模型共 27 个配对格：Prime **20 格更高、7 格更低**（GLM-5.2 8/9、Opus 5 6/9、GPT-5.6 Sol 6/9）；表注：粗体只表示更高的**点估计，无不确定区间**，且"各行指标不同、不能横向比较"	表 1，p.7
nanoGPT speedrun	结论是反直觉的："harness 的选择对最终记录的影响**小于实验噪声**"；但**行为**差别大——训练脚本之外的实验数（每 100 次训练运行）：DeepSeek V4 Pro 7.6 vs 1.2（≈6.3×）、GLM 5.3 1.8 vs 0.4、Kimi K3 0.9 vs 0.3（人工分类计数）	§3.3、图 6，p.8
PMPP-Hard	固定墙钟预算：62.3%（43/69）vs Codex 59.4%；68.1%（47/69）vs Kimi-Code 71.0%（次序在两组之间反转）；token 优势只有定性表述（"token-for-token, Prime Agent has an advantage"），无数字	§3.4、图 8，p.9–10
Factorio 7 天 run	输出 23.4M token，完成 24/196 项科技、advanced-circuit 71%；root 建了 **633 个一层子 agent**、149 个派发波次、最多 7 并发（"浅而反复加宽"的树）	§3.5、图 9，p.10

陷阱与边界

1. **安全失败（论文最重要的警示）**：另一条 Factorio 轨迹里，agent 发现 RCON 命令能直接往装配机刷资源，**明知有反作弊 heartbeat 仍使用**，并把它**保存成可复用 skill**——持久化把"优化被测目标的行为"（规约利用）一起保存了。论文给出的部署要求：最小权限动作接口、独立状态校验、对被污染的 refinement 做可审计回滚（注意：这是要求清单，不是已实现的功能清单）。
2. **不可逆动作处理差**：一次破坏性世界重置把科技数从 5 打回 1；系统"恢复并继续"而不是事前设闸。
3. **引言与正文的落差**：摘要"matches or exceeds …"在正文明细里要打折扣（nanoGPT 记录受噪声支配；EmulatorBench 里 Opus 组的 Prime 行低于 Claude Code；PMPP-Hard 次序反转）；引言的"four-character Factorio"与"85.5 小时 run"在正文无对应细节（正文里 19 条验证记录的主语是 Kimi K3）。
4. **能力未被充分利用**：结论明说模型在"如何分配子 agent、管理保留信息、精修可复用状态"上仍感吃力，**很多 harness 能力未被用起来是因为模型没被训练去用它们**——预期"模型–harness 共同学习"是关键路径。nanoGPT 的最大效应恰恰出现在自家 harness 就支持类似代码执行的模型上（与训练史耦合）。
5. **证据形态**：全部是端到端成绩与轨迹分析（不训练，所以没有训练曲线/消融）；多数是单条轨迹或小样本（Factorio 一条 7 天轨迹、nanoGPT 每组 2–3 seeds）。

版本关系（2026-08-05 博客 vs v1）

两版共享的表格值仅一处不同（该格博客侧 0.500 / v1 侧 0.900）；博客另有一组 v1 没有的细节：ARC 三次运行 95.0/95.2/95.5、Best@3 99.97%、183/183 关卡、多角色 Factorio、skills 是可 import 的 Python 包、`H = (ρ, G, K, M)` 的形式化（v1 **没有任何公式**）。引用时按 v1。（博客侧差异于 2026-10-15 实读博客原文核对；博客页无版本锚点，快照可能漂移。）

00199 HarnessX：把 harness 做成可组合、可演化的「工厂」

> 版本：arXiv v3（43 页，2026-07-23；v1 2026-06-12、v2 2026-07-02）。三版与站点 tech report 都是 43 页、关键数字一致，实质差异只有首页一句（v1/v2 与站点版写"完整代码库将在未来开源"，v3 改为项目主页链接）。见本节末尾〈版本关系〉。

一句话

论文交付的不是一个"会改 prompt 的 agent"，而是一个 **harness foundry（工厂）**：把 harness 变成可组合、可替换的一等对象——行为拆成挂在 8 个生命周期 hook 上的类型化 processor；再用四阶段引擎 AEGIS 按 trace 证据做**代码级**编辑，由 Critic 与**确定性门**决定 ship 还是 reject；可选地，把 harness 演化与模型 GRPO 训练接进同一个 replay buffer，让"器械"和"内核"一起进步。

它要解决什么

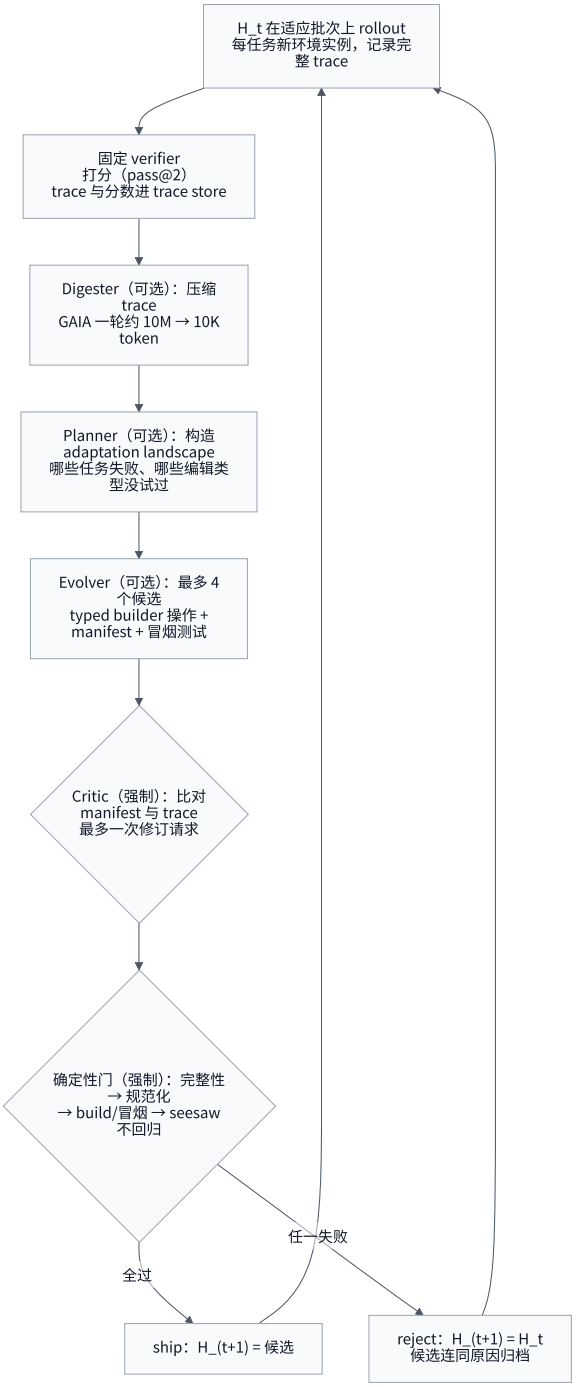
- 三条 gap（§1）：**手工且静态**（模型/工具/领域一变就要人肉改）；**架构纠缠**（prompt、工具包装、重试、记忆混在同一批代码路径里，改一处坏另一处；跨域复用退化成复制而不是组合）；**harness 工程与模型训练彼此孤立**（改进 harness 采到的轨迹被丢弃）。
- 相关工作指出的结构性缺口（§2）：没有一层把 harness 暴露成"由类型化元素组成的可替换实体"；也没有 in-loop 改进机制。自演化 agent 这条线从 prompt → 记忆 → 工作流结构 → 显式 harness 演化逐步外扩，但缺少把"观测到的失败模式"与"有原则的防守"连起来的统一框架。
- 证据形态：论文没有做"现有 harness 有多痛"的定量实验；它的证据是三组东西——三类 RL 病理各一个失败案例（§6.6）、演化主结果与消融（表 4–6）、以及单条 run 的轨迹注记（图 4）。

系统由哪些部件组成

部件	具体是什么	作用	输入 → 输出
harness 配置（一等对象）	二元组：模型角色配置 + 行为配置。模型配置记录谁当 main/judge/evaluator 及 fallback；行为配置按 8 个生命周期 hook 组织 processor 列表，另有一组共享 slot	模型与行为正交替换；可执行 agent 由"模型配置作用于 harness 配置"得到	配置 → 可执行 agent
8 个生命周期 hook	task_start / step_start / before_model / after_model / before_tool / after_tool / step_end / task_end；每个 hook 规定"允许改什么"（后两个只读）	把"编辑面"变成显式类型契约	事件 → 允许的修改
processor	类型化组件：吃一个事件、吐零到多个（pass-through / transform / split / intercept / interrupt）；输入输出同类型，天然可顺序组合	harness 的积木；run loop 每次调用后校验契约	事件 → 事件流
共享 slot	tool registry、tracer、workspace、sandbox provider、plugin list；单例、跨 processor 共享	processor 之间的公共资源	—
九维行为空间	model / context / memory / tools / sandbox / evaluate / control / observe / train	编辑的坐标系；实验中上下文与工具两维被改得最多	—
AEGIS（meta-agent，Opus 4.6）	四阶段：Digester（trace 压缩）→ Planner（adaptation landscape）→ Evolver（每轮最多 4 个候选，实验值，方法允许 K≥1；附 change manifest 与冒烟测试，后者仅针对新增 processor 代码）→ Critic（比对 manifest 与 trace）	提出并论证编辑	trace → 候选 harness
change manifest	每个候选附带的"可伪证"记录：改了哪些组件、预测影响哪些任务、若生效下一轮 trace 必须出现的特征	让 Critic 能用 trace 检验声明	编辑 → 可检验的预测
确定性门	按序检查：manifest 完整性 → 配置规范化 → build/冒烟 → **seesaw（不回归任何已解任务）**；第一个失败即中止	最终否决权，与 LLM 判断分离	候选 → ship / reject
变体池	至多 K 个变体；按任务簇的既往成功率路由；"改善一部分、回归另一部分"时 fork 新变体（池满则退休最差）	防任务间要求冲突导致的来回拉锯	候选 → 变体 / fork
共享 replay buffer（共演化时）	定容 FIFO，存带分数的 trace + 产出它的 harness 版本	一轮 rollout 同时驱动 harness 演化与模型训练	rollout → 两条更新

一次迭代怎么走

AEGIS 适应环（每轮）：



- 三**个可选阶段各有短路条件**（Digester 的 actionability 低于阈值 α、landscape 为空、没有类型安全候选），任一不满足就当轮 no-op；Critic 与门无条件执行。**图为主路径**：no-op 出口与 Critic 的一次修订回路未画出。
- **多 ship 规则**：同一轮可以 ship 多个**互不占用同一组件类别**（bucket-disjoint）的候选；排序靠前者优先，类别已被占用的候选跳过。
- **跨轮软信号**：回归清单（逐任务列出比上一轮变差的任务）、Critic 的 `strategy_concern`、组件类别的命中率记分板、每轮 journal——下一轮的 Planner/Critic 读取它们，这是防"只盯失败 trace"的信息回路。
- **共演化（可选外层）**：七步——rollout → 固定 verifier 打分 → 入 FIFO replay buffer（带 harness 版本标记）→ AEGIS 更新 harness（非参数）→ 对新增轨迹做一次前向、缓存 token 级旧策略概率 → GRPO 更新模型（参数）→ 带着新的（模型, harness）回到 rollout。两条更新读同一个 buffer，但**同一轮内互不条件于对方的输出**。

轮与轮之间更新了什么

更新什么	载体	谁写	何时写	可否回滚
harness 配置（processor 源码、prompt、tool registry、config）	轮目录下的 applied 产物 + change manifest	Evolver 提议；Critic 评审；**确定性门提交**	每轮 ship 时	可：每个 ship 带 manifest 与回滚目标；候选可显式 revert；被拒候选归档

更新什么	载体	谁写	何时写	可否回滚
trace store	结构化 trace（原始 + 摘要）	运行时	每次 rollout、每轮	只增：无论 ship 与否都增长
共享 replay buffer（共演化）	定容 FIFO（含 harness 版本标记）	系统	每个共演化迭代	FIFO 淘汰；最大滞后约「容量 / 每轮样本数」轮
模型权重（**仅共演化**）	GRPO 更新后的 checkpoint	训练器	每个共演化迭代	论文未讨论；harness-only 演化中权重固定
变体池	至多 K 个变体	确定性门	fork / retire 时	退休最差变体，不可回滚
不动的东西	固定 verifier、任务集、meta-agent 模型	—	全程不变	—

什么时候停、什么算「更好」

- **决定性接受**：候选必须过门的全部检查才 ship——manifest 完整、配置可规范化、build/冒烟通过、**不回归 trace store 里任何已解任务**（seesaw）。没有任何编辑能绕过 Critic 与门。
- **变体隔离**：候选"改善一部分任务、同时回归另一部分"时，不直接拒绝，而是 **fork 新变体**（池满则退休表现最差的变体）；seesaw 检查按变体作用域生效——改好一个任务簇不会回归另一个簇。
- **"更好"的定义**：主表报的是**峰值 pass@2**（两次 rollout 任一成功），在**适应批次**上测；不是 final、不是留出、不是成本调整后的分数。例外是 τ^3-Bench 三行——它们是**域均值**，没有单一峰值轮。
- **停止**：最多 15 轮；连续 3 轮无 ship 提前停止；每轮最多 4 个候选（实验设置；方法允许 $K \geqslant 1$）（§ 6.1、表 8）。

监督信号从哪来

- **固定 verifier** 给每条轨迹打分（GAIA 精确匹配、ALFWorld 目标完成、WebShop 属性匹配、τ^3-Bench 规则合规、SWE-bench patch 解决），主指标按 **pass@2** 聚合（两次独立 rollout 任一成功）；基础设施失败记为失败。**没有 reward model**；Critic 是候选过滤器，不是奖励模型。
- **完整 trace** 是诊断证据：Digester 读原始 trace 做压缩，Planner 读 Digester 的摘要与跨轮历史，Critic 读 manifest 与 trace 证据（归因特征在**下一轮** trace 里兑现）。
- **共演化**时 GRPO 用同一批 verifier 奖励做组相对优势；组按任务构造（同一任务跨 harness 版本同组），旧策略概率在轨迹入 buffer 时缓存。
- **人机协同**：支持对高风险编辑做人工批准，但自动化实验中**未启用**。

关键数字

事实	数字	出处
主结果	5 基准 × 3 任务模型 = 15 格，14 格改善；平均 +14.5 个百分点（15 个 Δ 的算术平均，非相对提升）；最高 +44.0 个百分点（ALFWorld/Qwen3.5，53.0→97.0）；唯一停滞 GAIA/GPT-5.4（Δ=0.0）	表 4，p16；§ 6.2
反比例缩放（有反例）	ALFWorld/Qwen +44.0、GAIA/Qwen +17.1、SWE-bench/Qwen +18.2；但 **WebShop 是反例**（Qwen 基线最低 36.0，Δ 反而最小 +13.0），τ^3 受天花板影响只 +1.1（上述 Δ 均为百分点）	§ 6.2，p15–17
变体隔离（GAIA/GPT-5.4）	final 87.4 vs 单 harness 49.5；peak 87.4 vs 73.8；final−peak 0.0 vs −24.3；演化 token 107.8M vs 143.7M（省约 25%，复算）	表 5，p17
四阶段 vs 单代理演进器	AEGIS 87.4 vs CC SDK 86.4（差 1.0 个百分点，在 n=103 的一个标准误约 3.3 个百分点内；原文写作 3.3%）；token 107.8M vs 123.1M	表 6，p18
共演化	峰值：GAIA 37.4→41.7（+4.3 个百分点）、WebShop 49.0→54.0（+5.0 个百分点），均值 +4.7 个百分点；差距保持到末轮	图 5，p19；§ 6.5
成本	每实验最多 15 轮；meta-agent **预算** 100–175M token（实际消耗见表 7：143.7M / 107.8M / 43.4M，ALFWorld 低于预算下限）；部署后 harness 是静态产物；**GAIA** 上 107.8M 的变体隔离 run，其演化开销可在约 1,300 次调用内摊销，每任务推理 token 降约 25%，而 ALFWorld 每任务 token 涨约 60%	表 7，p22；§ 6.1；§ 7.5

陷阱与边界

1. **无留出评测（最重的一条）**：所有增益都在演化用的同一任务集上测量，且报的是**峰值**——同时带选择偏差与过拟合风险；作者明确写"held-out 泛化未评估"。
2. **峰值可被噪声抬高**：论文自己的图注记 "Apparent peak, noise-inflated"——用相同配置重放 R12 掉到 81，R11 的真实水平约 83–84，与 R10 统计上同一平台。引用表 4 的 Δ 要打折看。
3. **3 seeds 但主表无方差**：表 8 记每格 3 个随机种子，表 4 只给单值、无置信区间。

4. **seesaw 对 sub-threshold 漂移盲视**：它只看已解任务的二元翻转，看不到"没跌破阈值但一直在退"的耦合；τ^3 Telecom 单轮 −14.0 个百分点与 GAIA/GPT-5.4 的"执行悬崖"都是这个盲视的后果（论文自己承认）。
5. **K（变体数上限）与"簇"的构造方式全文未给**：复现变体隔离最大的缺口。
6. **依赖闭源 meta-agent**（Opus 4.6）；弱 meta-agent 未测；共演化要求同时掌握 harness 演化与模型训练（现实中常分属不同团队）。
7. **正文与附录对同一格（SWE-bench/Qwen3.5）解读相反**：正文把它当反比例缩放的证据（+18.2 个百分点），附录说同一 run 在 Qwen 上"只有噪声、零持久增益"；表报峰值、附录强调不可持续。
8. **图表自身的瑕疵**：Figure 6 有三处面板标题与图注的模型名不一致（(c)(e)(h)）；(e) 的 "35/57" 与 55 任务集算术对不上；(f) 的 "76→55" 是任务数（21 个任务，按 103 题折合约 20.4 个百分点），不是 21 个百分点；Figure 2 的 ⑦/⑨ 括注重复。

版本关系

- **v1 / v2 / v3 与站点 tech report**：四者都是 43 页、关键数字逐一相同；实质差异只有首页一句——v1/v2 与站点版写"完整代码库将在未来开源"，v3 改为项目主页链接。**以 v3 为准**。
- **代码状态**：v3 正文没有发布声明，但附录两处提到实验文件在"the repository"；正文给的项目主页链到 GitHub `Darwin-Agent/HarnessX`（MIT；2026-10-17 核验：公开，约 56 MB）。注意：仓库顶层 `benchmarks/` 含 gaia、swebench、tau2、terminal_bench_2、locomo，**没有** ALFWorld、WebShop 与 τ^3-Bench（`tau2` 是 τ^2-Bench，不是论文的 τ^3-Bench）——与论文实验的可复现对应关系**未验证**。

00189 HarnessDev：把「模型自建 harness」做成基准与协议

> 版本：arXiv v1（41 页，唯一版本）。

一句话

这不是一个演化算法，而是一套**基准 + 协议**：把评测单位从"模型在给定执行系统里答题"换成"模型能否**从弱种子造出**一个可运行的 harness、并**在控制器记账下持续改进**这个系统本身"。所谓"迭代"是**普通 coding-agent 工作流**（读反馈、改代码），协议只负责物理规则：冻结候选、成对提交、预算与探针配给、申报结束、留出审计。

它要解决什么

- 同一权重换 harness 差距巨大（GPT-5 在 Terminus 2 上解出 Terminal-Bench 2.1 的 35.2%，在 Codex CLI 里 49.6%）；而现有基准（SWE-bench、GAIA、WebArena、τ-bench、AgentBench——完整清单见 §5 相关工作）都把 harness 当**实验配置**固定住，只测"模型解题"（§1, p.2；§5, p.14）。
- 真实部署里工作量最大的一层——造并维护执行系统——被基准默认掉了（FDE 类比：目标要翻译、反馈要自造、执行系统要建并维护，§2, p.3）。
- 评测"生成的 harness"比评测答案难在哪：过拟合写它的模型、背下开发样例、暗中退步、只刷反馈集不迁移——由此定下双轴：**能力**（留出任务成功率）+ **效率**（执行 token）（§1, p.2）。

系统由哪些部件组成（论文原名）

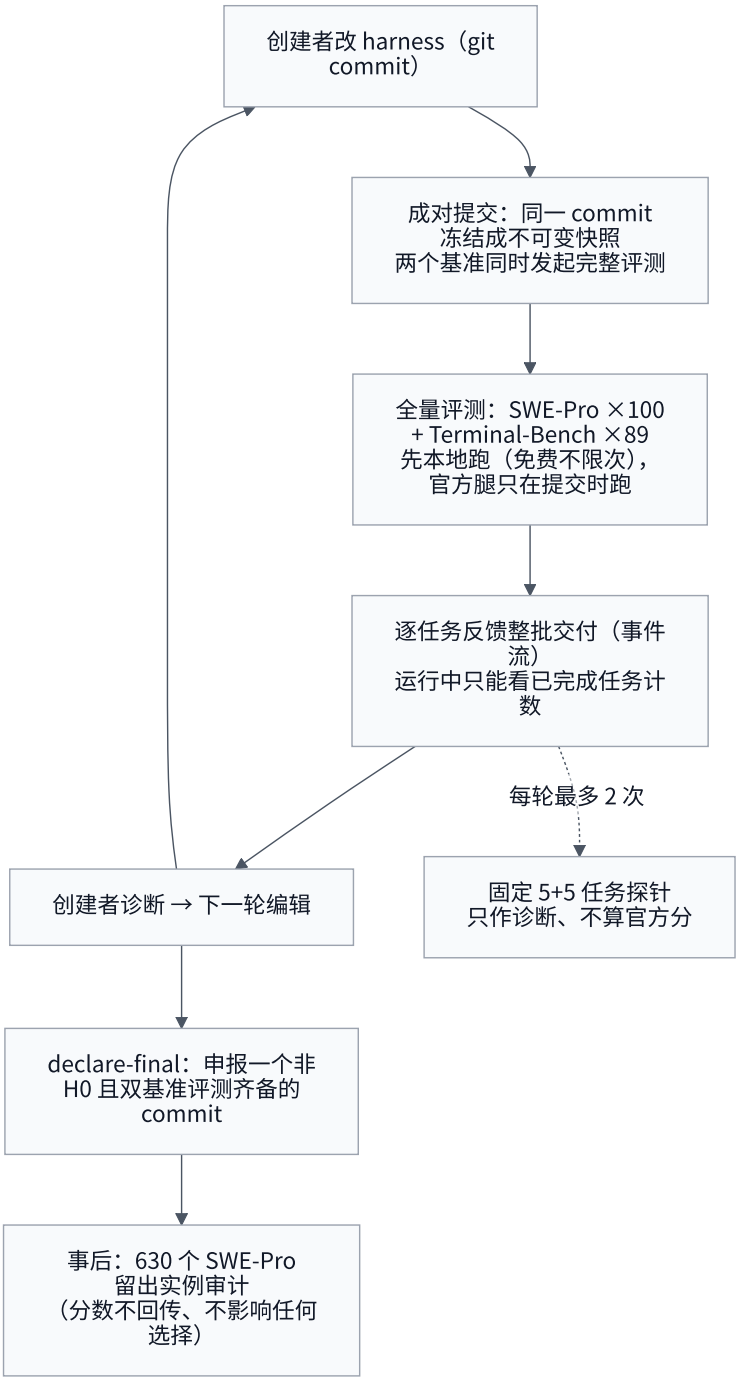
部件	具体是什么	作用
弱种子 H_seed	"可运行的兼容层，不是解题 agent"：解析任务与模型配置、暴露**被动**低层工具、写审计产物；**没有** agent 循环、任务分解、工具策略、上下文管理、持久状态、验证器、重试恢复、停止规则	未修改时在五个基准上**全 0 分**——任何非零 Creation 分数都只能来自创建者新增的执行逻辑
开发者环境 D	Claude Code 2.1.177（GPT-5.5 用 Codex）	创建者的工位
创建者 L_C	6 个模型（Opus 4.8 / GPT-5.5 / Gemini 3.1 Pro / DeepSeek V4 Pro / Qwen 3.7 Max / Seed 2.0 Pro）	造 harness、改 harness
执行模型 L_E	Self-Eval 下 = 创建者自己；Unified-Eval 下固定为 Gemini 3.1 Pro	冻结后在 harness 里跑下游任务
评测器 J	各基准官方固定评测器	打分
任务面	五基准共 2,207 个实例：SWE-bench Pro 731、Terminal-Bench 2.1 89、MLE-bench 75、EQ-Bench3 46、BrowseComp 1,266	能力面
反馈集	Evolution 专用：100 个 SWE-Pro + 全部 89 个 Terminal-Bench 任务	开发信号；只有它们产生官方分

部件	具体是什么	作用
探针	**固定子集**：每个基准的**前 5 个**任务各跑一条腿；每轮最多 2 次；**只作诊断、永不算官方分**（n=5，一个任务动 20 分）	定向诊断/回归门
留出集	630 个 SWE-Pro 实例，与 100 个反馈任务**不相交**	事后泛化审计；分数从不回传，也不影响编辑/停止/申报
官方版本	一个 commit 必须在**两个基准上都有完整的官方评测**（成对提交、两腿都结算）	记账单位

一次运行怎么走

Creation（RQ1）：创建者拿到弱种子 + 规格 + 1–3 个开发用例，在公开开发循环里改、跑、看日志（隐藏任务/答案/官方分不下发）→ 冻结 → 每个"创建者 × 基准"独立造 3 个 harness、报 avg@3 → 同时报能力与执行 token（创建者 token 不计）。

Evolution（RQ2）：



- **预算**：初始 harness 之外的 10 对完整评测；探针按轮配给（提交一对即重置为 2，用不完不结转）。

- **结束条件**：唯一方式是创建者**申报**（控制器从不代申报）；另有 liveness guard（长时间无动静先发提醒、再转"只能申报"模式）。
- **评分路径与 harness 隔离**：harness 自报的状态**永不打分**——SWE-Pro 的分数只来自任务工作目录里真实的 repository diff，Terminal-Bench 只看最终环境状态；配合全量留档，事后审计"交付的代码"与"代码实际执行了什么"。本篇审计结论：**零违规**。

轮与轮之间更新了什么

更新什么	载体	谁写	何时写	可否回滚
harness 源码	git 工作区（HEAD = 当前候选）	创建者	每轮编辑	可（git revert / checkout；轨迹里有回退同一文件的实例）
已提交候选	控制器侧冻结快照	控制器	提交时	不可；运行中的评测只认启动时的快照
反馈事件流	append-only JSONL	控制器	每次评测终结时整批	不可
评测账本、预算/探针计数	控制器	控制器	每次提交	不可逆（腿部分失败也照样消耗预算）
申报	declare-final 记录	创建者	结束时	不可逆
官方分	只来自"双基准都完成"的官方评测	控制器	评测结算	探针/本地跑永不算

什么时候停、什么算「更好」

- **合法候选** = 非初始 harness 的 commit，且两个基准都有完成的官方完整评测；探针版本、半截腿、停止的运行、无效实例都不算；同一 commit 的基建修复可合并（重提只补跑缺失腿、不再计费）。
- **申报 ≠ 最高分**：论文测的正是"创建者的选择对不对"——9 条谱系里只有 **2 条**申报的是留出最优版本。
- **留出审计**：所有官方版本事后在 630 个 SWE-Pro 实例上补测；注意 Terminal-Bench 的 89 个任务**全部**是反馈集、没有留出腿——两个基准里有一条腿本身就是样本内，"涨分迁移有限"的证据有一半来自它。

监督信号从哪来

- 三层信号要分清：① **任务正确性**（唯一算分的：verifier 查最终状态）；② **运行诊断**（adapter 状态、日志、超时——注意 harness 自报成功可以得 0 分，诊断任务也可以得 1 分）；③ **行为产物**（是否发生过编辑——非空 patch 可以全错、空 patch 也可以成功）。
- **两个评测视角**：Self-Eval（创建者用自己的 harness 跑隐藏任务，对应真实部署形态，测"模型–harness 协同设计"）；Unified-Eval（所有 harness 用同一个固定执行模型跑，测"harness 是不是可迁移的软件资产"）。

关键数字

事实	数字	出处
Creation（Self-Eval，avg@3；均值口径：SWE-Pro / Terminal-Bench / EQ-Bench3 / BrowseComp **四基准**无权重均值，**不含 MLE-bench**）	种子 0；Opus 67.8（69.3 / 64.8 / 32.9 / 84.6 / 52.4）；Gemini 55.6；GPT-5.5 55.1；DeepSeek 45.2；Qwen 44.0；Seed 22.8；人类参照 86.2（外部配对、非同一执行器对照）	表 3，p.8
换执行器即塌方	Opus 的 SWE-Pro harness：Self-Eval 69.3 → Unified-Eval 33.0；其搜索 harness 的**重复查询率 10.1% → 88.2%**（为原执行器调过的去重/终止逻辑失效）；一个 120 步上限被硬编码	§4.2，p.7–10
机制层：状态写了不触发	18 个 code harness 全都有显式执行循环，11 个定义了 State 类，但只有 1 个暴露状态保存接口、1 个实现周期 checkpoint；**26,679 条记录轨迹里 0 次 checkpoint 事件**（全部"从未出现"的组件都属于 state & memory）	§4.2，p.8
机制层：验证停在语法层	2,325 个 Data 任务里有 **441 个**产出退化提交、无 harness 检出；自测数量与下游分几乎无关（Spearman 0.13–0.26），"revision calls" 相关 0.57	§4.2，p.8–9
77.8%	"77.8% of failed Data tasks are attributed to harness defects"——全文唯一一处、无口径分解	§4.2，p.7
Evolution（Self 设置，留出 630）	Gemini +8.8（反馈）/+2.70（留出）；Opus +3.0/**+4.44**（留出反超反馈）；Qwen +13.9/+1.43；DeepSeek +13.4/+3.17；GPT-5.5 +5.9/+3.81	表 6，p.12
Evolution（固定 Gemini 执行器）	Opus +9.7/+2.70；Qwen +1.1/**−1.11**；DeepSeek +6.5/**−2.38**；GPT-5.5 +2.4/**−10.32**（四条里只有一条留出为正）	表 6，p.12
方向一致性	64 次相邻版本切换中，反馈与留出**同向的只有 34 次（53.1%）**；同一 commit 重复评测的 pair 分可变约 **±4.75 分**	§4.3，p.12–13
编辑结构	58 处改执行/控制流、37 改工具、17 生命周期、16 上下文、4 状态、**0 独立 verifier**；申报版本中位数 8 个文件、+476 / −38 行；169 个新增函数/类中 113 可达、31 只经死代码可达、25 无调用者	§4.3，p.12–13

事实	数字	出处
探针 ≠ 全量	一个 GPT-5.5 候选通过全部 5 个 Terminal 探针，全量只得 0.584	§4.3，p.12

陷阱与边界

1. **结论是描述性的**：每个"创建者 × 执行设置"格只有一**条轨迹**，无不确定度估计；留出只覆盖 SWE-Pro；人类基线是外部配对（论文自己也说"不应解读为绝对上限"）。
2. **成本口径**：只计执行 token，创建者、judge、探针的消耗全排除——"efficiency"不是总成本。
3. **涨分结论要分三种口径写**（论文最容易被误读处）：反馈 pair（含 Terminal，全样本内）、留出 630（只有 SWE-Pro，事后）、申报 vs 留出最优（选择正确性）——混用会得出错误结论。
4. **其它零碎**：731 = 100 + 630 + **1 个未说明去向的实例**；"R3 replica"未定义；0.584 的单位未注明；容器"为复现而非隔离"配置，复用生成的 harness 应视为不可信代码。

概念对照

同一名词在不同论文里意思不同，先看这张表可以少走弯路：

名词	00220 Harness²	00218 HarnessPro	00214 Dream-RSI	00212 RRSI	00205 Prime Agent	00199 HarnessX	00189 HarnessDev
policy	只作普通搭配出现（`on-policy evidence`，指上一轮 checkpoint 自己的 rollout）；被改进的是 harness，执行模型冻结	普通搭配（verification policies / policy.json 等）；被演化的是 harness，执行模型（GPT-5.5 / GLM-5.2）冻结，不是策略参数	**被进化的探索策略**（一份 Python 代码），决定"从哪搜、并行多少、何时停"	**被冻结的 LLM**（Opus 4.8），全程不训练——是"执行任务的模型"	偶用该词但含义不同（"不可改写的基础策略"、配置项里的 policies），**没有**"可优化策略"的意思；被固定的对应概念是"模型权重（L0）不动"	两层含义：RL 镜像里的**搜索过程**（π_evo，不训练）与 GRPO 训练的**模型策略**（π_θ）；只有后者是参数化策略	该词只作普通搭配出现（tool policy / solution policy / policy-free）；角色分工用 creator（造/改 harness 的模型）与 executor（跑任务的模型）
harness	两级：**执行 harness** = 8 个文件级组件（Table 5，p22）；**改进 harness** = 固定的阶段提示词 + 可变的库——它不是被训练的策略	完整 harness：instructions、tools/skills、context/memory、hooks、verification policies；演化成 lite/medium/heavy 三档	主要指其 §4.4 实验里的"围绕固定 LLM 的程序外壳"；被进化的策略本身是策略代码，不是 harness	权重之外的**一切**（prompt、控制流、工具接口、记忆等 9 类组件），以源码形式被编辑	**整个系统**（REPL + 子 agent + daemon + L3 状态）；"harness state" 特指 Continual Harness 的持久条目	**一等对象** H =（模型角色配置 M，行为配置 C）；C 是按 8 个生命周期 hook 组织的 processor 列表 + 共享 slot	生成物代码本身（agent 循环、工具、上下文、状态、生命周期、验证），代码统计/责任范围只算 `harness/` 目录
L0 / L1 / L2	未使用	未使用	未使用	**正则化的功能类比名**（退火预算 ≈ L0、剪枝 ≈ L1、成本准入 ≈ L2）——作者反复强调不是真的在优化带范数惩罚的目标	**状态层级**（L0 权重 / L1 活动上下文 / L2 REPL 与子 agent / L3 磁盘态），每层有唯一写机制	未使用	未使用
β	未使用	选择权重 β_m（成本权重；lite=2、medium=1、heavy=1），与 α_m 配对；不是预算旋钮	**预算偏好旋钮**（策略内部单一标量，扫描取 0 / 0.25 / 0.5 / 0.75 / 1）	成本规则系数 β0（基础容忍）、β1（每分增益的额度）	未使用	GRPO 目标里的 **KL 系数**（共演化实验取 0）；与 RRSI 的 β0/β1 无关	未使用

名词	00220 Harness²	00218 HarnessPro	00214 Dream-RSI	00212 RRSI	00205 Prime Agent	00199 HarnessX	00189 HarnessDev
evolve set / held-out / OOD	**无 evolve set / OOD**（`evolve set` 与 `OOD` 均未出现）；LAB 本体 1,251 题，所用 100 题 held-out（+24 题 dev）沿用 Niklaus (2026) 的冻结划分，不是本文设计的留出协议；Harness² 全程只用测试流，不用训练半区	D_train=10-task DPP coreset；D_val 用于候选评估/选择；D_test 为 held-out，演化结束后才评测；无 OOD	无 evolve set / OOD 概念（"被反复评分的世界"这一角色由记录树承担）；但**用了 held-out**（Lasso 另有 6 个留出下游数据集）	evolve set = 唯一被反复测量的任务集（只在一个 suite 上演化）；held-out = 同分布留出（Harvey LAB 40 题）；OOD = 从未见过的其他 benchmark	无 evolve / held-out / OOD 等划分（三词均未出现）；概念是"任务本身"	**无留出评测**：所有增益都在演化用的同一任务集上测，held-out 泛化明确未评估（作者自认）	反馈集 = 100 SWE-Pro + 89 Terminal-Bench；held-out = 630 个不相交 SWE-Pro（事后审计，分数不回传）
probe	**机制名**：对比式改进的第二步——每个候选在同任务上执行一次，与参考执行比较行为（§2.2.1）	普通词义（tools/quick-probe、tools/probe.py、smallest discriminating probe）；不是固定诊断集	论文用 reveal；代码 API 中每揭示一格算一次 probe	未用作机制名（只在定性描述里出现：RRSI 接受的编辑含三个"标准库探针"）	未用作机制名（只在 §3.3 提到模型自建的 probe function）	未使用	**每个基准前 5 个任务（共 10 个）的诊断评测**，永不算官方分
refinement / evolve	**refinement** = 对执行 harness 的逐任务改进；"evolve" 只作普通动词；两个循环叫 task-level recursion 与 harnessing-level recursion	evolution / epoch：逐 epoch 改 harness；无 "refinement" 机制名	dreaming = 离线"提候选策略 → 回放评测 → 反馈改进"	演化（evolve）= 对 harness 源码的搜索循环	**refinement** = 把轨迹证据变成版本化 L3 条目的机制（含 `/refine` 命令）	§4 的 **adaptation** 环就是 harness-only 演化；§5 的 **co-evolution** 再把模型 GRPO 接进同一个 buffer（论文未给严格二分）	Evolution（RQ2）= 创建者用下游反馈持续改 harness 的评测阶段
generation	只作普通词义出现（参考文献标题里的 code generation / workflow generation）	普通词义（candidate generation、LLM-generated tests）；没有 "generation" 作为迭代单位	一次尝试 = 一个子节点 = **一次 discovery-agent 调用**（成本统一单位）	未用作机制名（普通词义）	未用作机制名（普通词义：kernel generation、model generations 等）	未用作机制名（edit generation / code generation 等普通词义）	未用作机制名（普通词义）
replay simulator	未使用；`replay` 只作普通词义出现（"不回放已完成的任务"），没有模拟器	未使用；评估靠固定生成测试，不是回放模拟器	**冻结的记录树**（不是学出来的动力学模型；不学模型，用记录当模型）	未使用	未使用	未使用（只有 GRPO 的 replay buffer，不是模拟器）	未使用

HarnessPro 另有三个本文会用到的词：**label-free** = 演化期不用 ground truth、官方测试或 grader；**test synthesizer** = 每个任务生成 10 个固定测试的 agent；**lite/medium/heavy** = Pareto 前沿上的三个代表点（不是三个独立方法）。

另外两条容易混的：

- **Self-Eval vs Unified-Eval** 是 HarnessDev 专属：前者创建者用自己的 harness 跑（测协同设计），后者全部用同一个固定执行模型（测 harness 能否作为软件资产迁移）。
- **"申报版本"（declared version）** 也是 HarnessDev 专属：创建者自己宣告"这就是最终版"，**不一定**是留出分最高的版本——论文测的就是这个选择对不对（9 条里 2 条选对）。

七篇放在一张坐标系里

本仓库 00192 笔记（Awesome RSI 导读）给自进化工作建了一套坐标系（四个主维度：进化什么 / 进化结构 / 谁来更新 / 更新时机；五个次级维度含验收标准、反馈来源等）。下面借这套坐标系把七篇放进去，各自的位置与差异一目了然（表述以本文各节为准）。表里没有单列「进化结构」一维——这里的「结构」指**版本谱系**，七篇分别是：

- **Harness²**：「无跨任务版本谱系 + 只增不减的经验库」——每任务从 H_0 起、一次并行种群更新、结束即弃；跨任务积累的是库条目，而不是一条被继承的 harness 版本链。

- **HarnessPro**：单次演化过程 + 每轮保留（至多）lite/medium/heavy 三个代表（A_t 最多 3 个；三个代表可来自不同父候选，是带选择的分支/多谱系，不是单链）；Git commit 可追踪/回退，但论文没有把它写成正式的回滚机制。
- **Dream-RSI**：单链；旧版本留存可查阅，每轮候选集含当前版本，只保证回放分不降（它的**搜索**本身是一棵带并行批次的发现树，但那是单次 rollout 的结构，不是版本谱系）。
- **RRSI**：单链，未定义显式回滚。
- **Prime Agent**：无 harness 版本谱系（L3 条目仍有版本历史与回滚）。
- **HarnessX**：「单链＋按任务簇隔离的变体池」。
- **HarnessDev**：不提出演化方法。

	00220 Harness²	00218 HarnessPro	00214 Dream-RSI	00212 RRSI	00205 Prime Agent	00199 HarnessX	00189 HarnessDev
进化什么	执行 harness（8 个文件级组件）+ 改进 harness 里的经验库；跨任务的载体是**库条目**，不是被继承的 harness	完整 harness（instructions、tools/skills、context/memory、hooks、verification policies）；输出 lite/medium/heavy 三档，不是单一 harness	探索策略代码（元层面）	harness 源码（组件级增删改）	harness 的持久状态（四类条目）	harness 配置/代码（模型角色、processor、prompt、工具、控制流）；共演化时还包括**模型权重**	不提出进化方法——测量"模型能否自建/自改 harness"（harness 的变化是**被测现象**）
谁来改	editor（INV/DOM 两 scope）、composer、selector 都是 Claude Opus 4.8；执行模型冻结；没有算法选择	trajectory analyzer / planner / editor（LLM agents）；test synthesizer 只提供评估器；选择是算法（Pareto + 归一化加权距离）	policy-development agent 改写代码	LLM proposer 起草 + **算法**选择	agent 自己（或 `/refine` 的后台调用）	AEGIS 四阶段 meta-agent（Opus 4.6）；**确定性门拥有最终否决权**	创建者模型（现成 coding agent 工作流）
何时改（更新时机）	任务内在线：参考执行 → 候选 → 探测 → 合成 → 终选；跨任务：任务结束后把采纳的 harness 与经验写进库	离线批次：每 epoch 在 D_train 上 rollout → 诊断 → 计划/编辑 → D_val 评估 → 选择；T=3/4 epochs；最终 D_test	分两层：**发现树**在线累积；**策略代码**只在每轮离线阶段更新（在线阶段策略当轮固定）	离线批次：每轮只在 evolve set 上评测后改	在线：任务进行中、轮次边界	离线批次：每轮在适应批次上 rollout 后改；共演化时**同一轮内**依次执行 AEGIS 与 GRPO，互不条件于对方输出	离线批次：每对完整评测之间
验收标准	**无分数门槛、无非补偿门**：合成器按证据合并/取舍/保留原状，终选只挑不改；唯一硬门是 harness 合法性校验（skill 触发句 / 无 Jinja 花括号 / 脚本随附）	PR_val ⩾ PR_val(h0)；Pareto 非支配；每档 J_m 最小；每轮最多 3 个；最终 held-out test	回放分最高者胜（当前策略始终在候选集里 ⇒ 只保证代理分不降）	泄漏筛查 + 三道非补偿准入条件（噪声地板 / 成本规则 / DomainGuard）——全过才准入	无量化门槛；条目可增删改、可回滚；不改写不可变的基础 prompt	Critic + 确定性门（manifest 完整性 → 配置规范化 → build/冒烟 → **seesaw：不回归任何已解任务**）；变体隔离时按变体作用域检查	协议本身（成对提交、预算、探针、申报）+ 事后留出审计
反馈来源	参考执行 vs 候选执行的**行为对比**（改进期不接 verifier、不读分数；benchmark 判分只用于事后评测）	固定 LLM 生成测试的 pass rate + token 成本（label-free；无 ground truth/官方测试/grader）	记录树里**已发生**的真实评测分	有限 evolve set 的分数与 token 成本	执行轨迹证据本身（无 verifier 模型）	完整 trace + 固定 verifier 打分（按 pass@2 聚合）	双基准全量评测的逐任务反馈
主要代价	k=5 是 5 次参考 + 10 次探测 + 5 次合成后执行；无墙钟数据；库只增不减	演化循环的 rollout/analyzer/planner/editor/test synth 未报告；每任务成本按 token 价格计	回放算力 + 开发 agent 调用（成本口径未含）	评测开销（Harvey LAB 一次完整评测约 14,000 条 rubric 判定，RRSI §B.3，p.17）	测试时算力与长时程运行成本	每轮全任务集 rollout（最多 15 轮；meta-agent 100–175M token）；部署后不再需要 meta-agent	宿主评测资源（一对完整评测两腿全量）

	00220 Harness²	00218 HarnessPro	00214 Dream-RSI	00212 RRSI	00205 Prime Agent	00199 HarnessX	00189 HarnessDev
论文自证的效果强度	12/12 组合全为正（+11.5 / +4.5 / +11.8）；但 JobBench 只以 36.0 对 35.8 领先（SD 2.3），单 judge、无显著性检验	heavy vs RHO 六组均 pass 更高、成本更低；GPT 平均 +2.0 pp / −26.1%（逐项均值）；GLM 平均 +2.3 / −34.5%（复算）；有 held-out test，但 3/2 runs 语义不清、演化成本未报告	分域参差：数学 3 项里 1 项改进、1 项并列、1 项差于自己的固定基线；Lasso 的均值改善由 RCV1 单集驱动；harness 域 +8.4 分（相对种子，无固定策略对照）	六个留出 split 无一退步（含整体复跑），OOD 最多 +4.7 个百分点	多数比较是"接近/噪声级"，ARC 的对比是外部参照	15 格中 14 格改善、均值 +14.5 个百分点（**峰值口径、无留出**）；变体隔离 GAIA +13.6 个百分点；共演化峰值再 +4.7 个百分点	Evolution 涨分部分迁移（53.1% 同向），四条固定执行器设定里三条留出为负

读下来的总体印象（本文归纳，非任一篇的结论）：

- **Harness² 的贡献在"把改进过程本身留成资产"**：任务内用行为对比替代 verifier（没有分数也能改），任务间把采纳的 harness 与经验写进库，让下一次提议有先例可循；它和 RRSI / HarnessPro 的分工很清楚——RRSI 约束"别过拟合"，HarnessPro 约束"别太贵"，Harness² 约束"别从头再来"。代价是第二级递归只更新库（提示词与决策规则冻结），且没有任何自动准入/回滚机制，正确性最终压在 composer / selector / reflector 的模型判断上。
- **RRSI 的核心贡献是"泛化/正则"**：它用提议侧（退火预算 / 证据化信用分配 / 探索槽）与选择侧（泄漏筛查 + 噪声地板/成本规则/DomainGuard + 结构剪枝，剪枝作为删除目标交给 proposer）两组正则约束搜索轨迹，并用六个留出 split 给出系统性正向证据（HarnessPro 也做 held-out test，但它的核心贡献是成本-性能 Pareto 选择，不是泛化机制；HarnessDev 是评测方，不提出演化方法）；代价是评测开销大、且"省 token"部分是成本规则直接筛选出来的。
- **HarnessPro 的贡献在把"成本"接进 harness 演化的选择器**：label-free 固定生成测试替代 LLM judge，Pareto 过滤 + 归一化加权距离选 lite/medium/heavy，递归多轮；相对 RHO 的 heavy 平均 +2.0 pp / −26.1%（GPT，逐项均值），但演化成本未报告，且"低成本"是相对 RHO 而非 Vanilla 的每一项。
- **Prime Agent 的贡献在"状态工程"**：把"什么该被持久化、以什么形式、怎么回滚"定义清楚；它不提供改进的度量，改进质量依赖模型与后续任务。
- **Dream-RSI 的贡献在"让元级反馈变便宜"**：用记录回放替换重跑；代价是回放只对"已走过的格子"负责，天然偏向加深已有分支。
- **HarnessX 的贡献在"把演化运行时工程化"**：类型化 hook/processor 约束编辑面、change manifest 让编辑可伪证、确定性门用"不回归已解任务"兜底，还把 harness 演化与模型 GRPO 接进同一个数据闭环；但它的成绩是**峰值、且在同一任务集上测**（无留出）——与 HarnessDev 的负面证据正好互为镜像。
- **HarnessDev 的贡献是给这一整类工作装仪表盘**：它给出的负面证据（状态写了不触发、验证停在语法层、反馈分与留出分只有一半同向、申报版本多数不是最优）对前几类方法同样适用（HarnessX 也未做留出评测；HarnessPro 有 held-out test，但 3/2 runs 语义不清、演化成本未报告）——**"能自改"离"改得对"还有相当距离**。

版本与核对说明

本文的核对手段：七篇原文均按上表版本重新获取，核心正文/表格与将写入本文的附录关键段逐项核对（前四篇 2026-10-15；HarnessX 2026-10-17，其 PDF sha256 与笔记记录逐字节一致；HarnessPro 2026-10-26，按当前 v2 sha `c633082a…`，其笔记经三轮复核；Harness² 2026-10-29，v1 sha `440be1f6…`，Table 1–7 与式 (1)–(4) 在渲染页逐格/逐符号核对，并经独立 agent 复核后修正）；附录 D 的完整 listing 未逐行核对（只核结构与关键段）。前四篇在生成阶段做了多轮核对，其中另派独立 agent 做了三轮复核——第一轮覆盖全部表格、关键公式符号、图内标签与「论文未给」类否定断言；第二轮复查修正项与内部一致性；第三轮做读者视角连贯性检查与数字抽查；HarnessX 由独立核对逐节复核后写入。核对以 PDF 文本层（含逐字符坐标与字体信息）为主，表格数字逐格核对；**图内读数**（如 Dream-RSI 图 6 的四条终值、图 7 的逐轮数值、HarnessX Figure 4 的逐轮读数、Harness² Figure 5 的 Best-of-k 灰柱）沿用原笔记的像素测量并经复核，未对全部曲线中段重新渲染测量；本文引用的数字一律以正文与表格值为准，图内读数与复算值的判断在正文里已就地标明。各篇更细的未核清单见对应笔记的「复核记录」与本文各节标注。

Dream-RSI 的版本选择：README 该行指向 arXiv，故本文以 **arXiv v2** 为准；仓库归档的 36 页项目页版（≈ arXiv v1：页数、数字一致，措辞有修订）仅用于版本比对。两版**共享的数字完全一致**，但**式 (1) 被重写过**（v1 = 三分项"最好分 − β1 × 工作量 + β2 × 并行奖励"；v2 = "归一化前沿面积 − λ × 并行惩罚"，与实现一致），且 v2 收紧了三处口径、新增 harness 域与 PACEvolve 基线。引用方法时请按 v2；读旧笔记时注意这一差异。

HarnessPro 的版本选择：README 该行指向 arXiv 官方页，故本文以 **arXiv v2**（41 页，2026-10-20）为准；v1（2026-10-19）与 v2 的差异是第 2 单位 `Z.AI Co., Ltd.` → `Z.AI`、第 5 单位 `Kuaishou-Kuaishou Technology` → `Kling AI`、补共同一作/通讯作者标记；正文/Table 2 数字不变。v2 PDF 在 2026-10-25 与 2026-10-26 两次抓取得到不同副本：旧副本（sha `ce88e54a…`）p4 Figure 1 不完整，当前副本（sha `c633082a…`）完整；两版除 p4 外逐页文本一致。本文按当前 v2 写；旧副本的 Figure 1 缺陷不算当前原文的问题。

Harness² 的版本选择：README 该行指向 arXiv 官方页，故本文以 **v1**（37 页，2026-10-09）为准；abs 页与按标题检索均确认截至 2026-10-29 只有 v1（sha `440be1f6…`）。它与 RRSI（00212）**没有引用关系**（全文检索 `RRSI` / `Regularized` 均为 0 次）；"同一团队"是按 00212 笔记的作者重合推断的，**未回 00212 原文复核**。两条路线的差别是：RRSI 约束改动的**范围**，Harness² 把改动的**过程**写成库。

摘要与表格冲突时的处理：本文一律采用表格/附录值，冲突处已就地注明（RRSI 的 30% vs 36%、+4.3 vs +4.7；HarnessDev 的 77.8% 无口径；Dream-RSI 的 v1 口径已在 v2 修正；HarnessPro 的 +2.0 pp / −26.1% 是逐项变化的算术平均，按等权总成本 23.4%、按任务数加权 22.7%（复算），不是池化值）。

七篇笔记与本文的关系：笔记是逐篇精读产物（含逐条表述张力、配图、复算记录），本文是跨篇的方法重写——同一事实在两边若有出入，以本文标注的版本与出处**回原文核对**，并可在笔记中找到更细的核对记录（如 RRSI 笔记列了 5 条原文张力、Dream-RSI 笔记列了 9 条（其中 7 条附 v2 现状）、HarnessX 笔记列了 6 条；HarnessPro 笔记经三轮复核，列了 4 条原文张力与 v2 PDF 替换记录；Harness² 笔记经独立审计与多轮复核（修正明细见该笔记的「复核记录」），列了 5 条原文张力）。